

UNIVERSITAT AUTÒNOMA DE BARCELONA

MASTER'S DEGREE IN MODELLING FOR SCIENCE AND
ENGINEERING

FINAL MASTER'S THESIS

Optimizing Full Waveform Inversion Through the Implementation of a Spectral Element Solver

Author: Eduard González Yu

Supervisors: David Romero (CRM), Marc Jorba (CRM), Estela Jiménez (ICM)



June 2026

Acknowledgments

I would like to thank my three wonderful supervisors, Dr. Marc Jorba, Dr. David Romero, and Dr. Clara Estela Jiménez, as without their guidance this thesis would not have been possible. Their knowledge, support, and attention have been crucial throughout these past months. Outside of the thesis, they have also advised me on my future, which has helped me stay motivated.

I would also like to thank my family and friends, for supporting me and encouraging me to keep on working and educating myself.

Finally I would like to acknowledge Dr. Carmelo Puliatti and Dr. Vicent Costa, for introducing me to the world of research.

Abstract

Full Waveform Inversion (FWI) is the benchmark methodology in marine geophysics for obtaining high-resolution images of the subsurface. Within the FWI workflow, the forward modeling solver acts as the backbone of the method. Achieving robust and high resolution inversions requires not only a physically realistic and numerically accurate solution to the wave equation but also the geometric flexibility necessary to faithfully honor the structural complexity of the subsurface. Traditionally, FWI codes have relied on Finite Difference Methods (FDM) due to their simplicity. However, FDM solvers frequently introduce «staircase» artifacts when representing irregular topographies and interfaces, leading to numerical dispersion and inaccuracies in the inverted models.

This work presents the development of a solver based on the Spectral Element Method (SEM) designed to replace the conventional FDM engine in 2-D. By utilizing deformed hexahedral meshes and high-order polynomials (degrees 4 to 10), the SEM achieves «spectral accuracy», allowing for a representation of actual geology without the distortions inherent to rectangular grids. Unlike FDM, which requires a high Points-Per-Wavelength (PPW) count and high RAM consumption, the SEM guarantees accurate solutions with low PPW, significantly optimizing computational resources. This transition is critical for high-resolution FWI: by eliminating numerical dispersion and preserving wave phase over long distances, it ensures that physical model updates correspond to real geological structures rather than artifacts of the numerical scheme.

Contents

1	Introduction and Problem statement	1
1.1	FWI in Seismic Exploration	1
1.1.1	The Need for Accurate and Efficient Solvers	2
1.2	Geophysical Context and the Acoustic Approximation in Marine Exploration	2
1.3	Objectives and Outline of the Thesis	6
2	Mathematical Preliminaries	7
2.1	Vector Calculus	7
2.2	Partial Differential Equations	8
2.3	Finite Element Methods	9
2.4	Spectral Methods	11
3	The Spectral Element Method in 1-D	13
3.1	Weak Formulation	13
3.2	Domain Decomposition	15
3.3	GLL Quadrature	16
3.4	Local Interpolation and Shape Functions	17
3.5	Integration of the Weak Form	18
3.5.1	Elemental Mass Matrix	18
3.5.2	Elemental Stiffness Matrix	19
3.6	Assembly of the Global System	20
4	The Spectral Element Method in 2-D	22
4.1	Weak Formulation	22
4.2	Domain Decomposition	23
4.3	Local Interpolation, Shape Functions and GLL Quadrature	24
4.4	Mapping of Local indices to Global indices	24
4.5	Integration of the Weak Form	25
4.5.1	Elemental Mass Matrix	26
4.5.2	Elemental Stiffness Matrix	26
4.6	Mesh Algorithms	27
4.7	Perfectly Matched Layers	30
4.8	Global Assembly and Implementation Details	35
5	Validation and Results	36
5.1	Implementation in 1D	36
5.1.1	Simulation 1	36

CONTENTS

5.2	Implementation in 2D	38
5.2.1	Simulation 1	38
5.2.2	Simulation 2	40
5.2.3	Simulation 3: The Marmousi Model	43
6	Conclusions and Further Work	47
	Appendices	52
A	Algorithms	52
A.1	Pseudocode	52
A.2	Bilinear Interpolation	53

1 Introduction and Problem statement

1.1 FWI in Seismic Exploration

Full Waveform Inversion (FWI) [1, 2] is the gold standard methodology in seismic exploration and subsurface characterization due to its capability to reconstruct high-resolution models of the physical properties of the Earth (such as wave velocity c). Unlike other methods, FWI exploits the full information content of the recorded seismograms, including phases, multi-parameter reflections, and diffracted waves.

From an optimization perspective, FWI is formulated as a non-linear, poorly-conditioned Partial Differential Equation (PDE) constrained optimization problem. The primary objective is to minimize a misfit function, χ , which measures the discrepancy between the observed experimental data ($\mathbf{d}_{\text{obs}}$) and the synthetic data ($\mathbf{d}_{\text{syn}}$) generated via numerical simulation for a given vector of model parameters $\mathbf{m}$:

$$\chi(\mathbf{m}) = \frac{1}{2} \sum_i \int_I M_{R_i}(t) \, dt ,$$

where M represents the objective function (typically under a least-squares criterion), I the time interval, and the summation applies to all receivers R_i .

The optimization workflow is inherently iterative and follows a well-defined gradient-based scheme (e.g., Non-Linear Conjugate Gradient, NLCG):

- **Forward propagation:** Synthetic wavefields $u^s(\mathbf{x}, t)$ are generated by solving the governing wave equation under an initial or updated model parameterization.
- **Gradient computation via the Adjoint Method:** To avoid computationally prohibitive finite-difference parameter perturbations, which would require the explicit and direct calculation of the Fréchet derivatives (Jacobian matrix) by perturbing each model parameter individually, the adjoint-state method is deployed to evaluate the exact gradient of the misfit function (e.g., $g = \nabla_c \chi$). Rather than being a mathematical approximation, the adjoint method provides [1] an efficient formulation to compute the true analytical gradient for all parameters simultaneously. This is achieved by the temporal convolution of the forward wavefield u^s and an auxiliary adjoint wavefield ψ :

$$g(\mathbf{x}) = - \int_I \frac{\partial u^s(\mathbf{x}, t)}{\partial t} \cdot \frac{\partial \psi(\mathbf{x}, t_f - t)}{\partial t} \, dt .$$

The adjoint field ψ must be computed by back-propagating the data residuals (acting as adjoint sources, $\mathbf{f}(t) = -\nabla_{u^s} M(t)$) backward in time from the receiver positions.

- **Model Update:** A search direction $\mathbf{d}_i$ is constructed using the current and historical gradients to update the model parameters towards the global minimum:

$$\mathbf{d}_i = -g_i + \beta_i \mathbf{d}_{i-1} ; \quad \beta_i = \frac{g_i^T (g_i - g_{i-1})}{\|g_i\|^2} ,$$

$$c_{i+1} = c_i + \alpha_i \cdot \mathbf{d}_i .$$

1.1.1 The Need for Accurate and Efficient Solvers

The practical feasibility of FWI is entirely governed by the computational efficiency and numerical scalability of the underlying PDE solver. As detailed in the workflow (Figure 1), a single FWI iteration requires solving the forward wave equation for every single seismic source (shot), followed by solving the corresponding adjoint wave equation backward in time. In real-world exploration campaigns featuring hundreds or thousands of shots, and requiring dozens of optimization iterations, the wave equation must be solved tens of thousands of times.

Currently, standard industrial and academic implementations rely heavily on FDM due to their straightforward implementation on regular grids. However, FDM exhibits severe limitations when dealing with complex, non-planar geometries, sharp topographic variations, and high-frequency dispersion (numerical grid artifacts). In FDM, representing curved geological interfaces or surface topography on a rectilinear grid introduces a staircasing effect, which scatters waves artificially and degrades the accuracy of the synthetic data.

Consequently, there is a critical need to design and implement advanced numerical solvers that are both highly accurate and computationally efficient. This master’s thesis addresses this bottleneck by developing a forward solver based on the Spectral Element Method (SEM). This allows for the exploitation of unstructured meshes and high-order geometric approximations, significantly reducing the number of degrees of freedom required to accurately simulate wave propagation. Enhancing the solver’s algorithmic efficiency and optimization directly translates into reducing the massive computational time of the FWI inversion loop, rendering high-resolution imaging mathematically and practically tractable.

1.2 Geophysical Context and the Acoustic Approximation in Marine Exploration

Before introducing the standard mathematical and physical approximations in seismic imaging, it is essential to contextualize the inherent complexity of the Earth’s subsurface as the propagation medium [3]. Marine geophysical exploration uses active seismic sources (air guns) to inject acoustic energy into the water

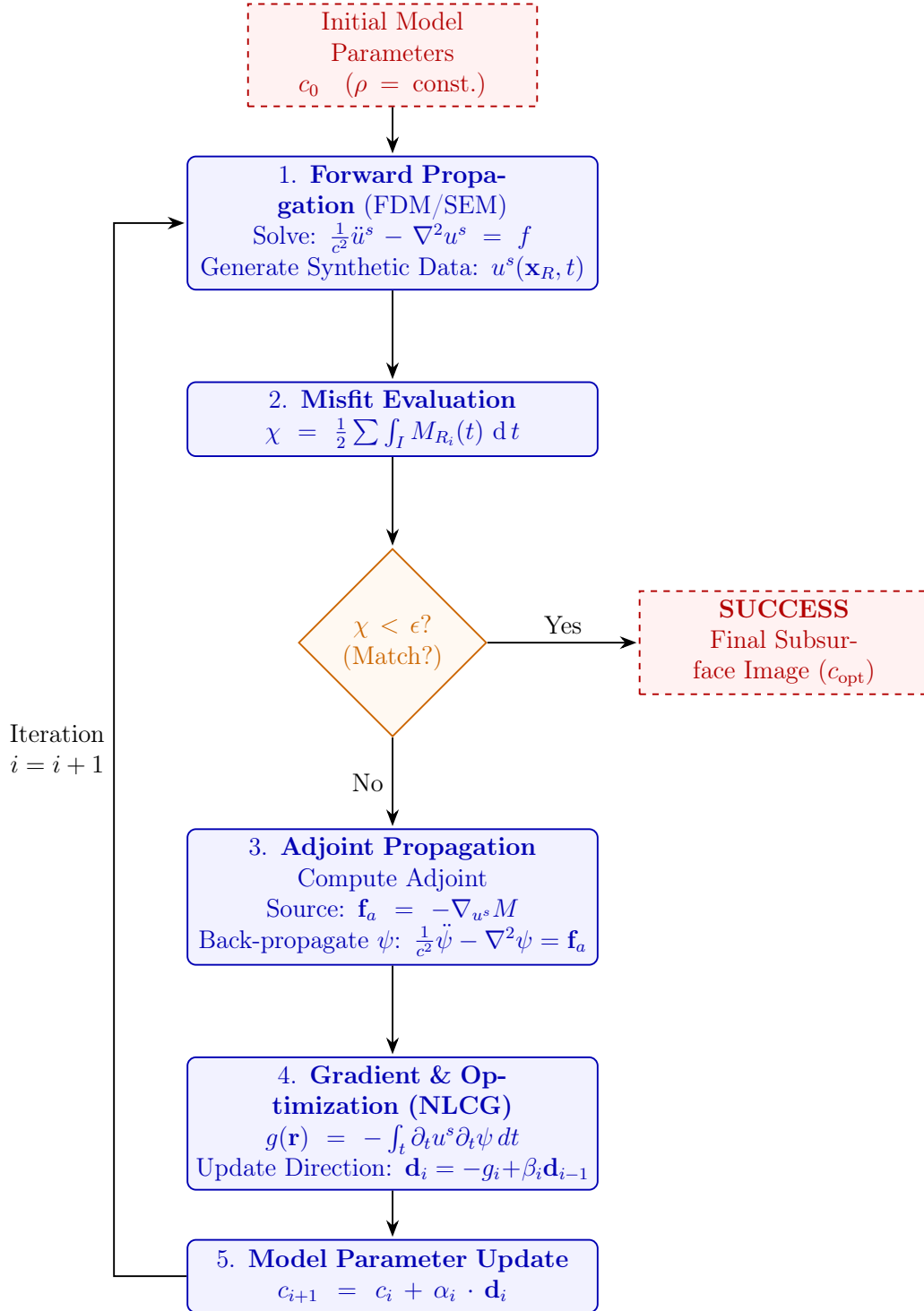


Figure 1: Summary of the workflow from Full Wave Form Inversion. The core numerical solver is called at steps 1 and 3 inside the iterative inversion loop, making its efficiency paramount for the overall computational cost.

column, which subsequently penetrates the geological strata. As seismic waves propagate through the subsurface, they undergo continuous reflection, refraction, and scattering at boundaries where the mechanical properties of the rocks change.

Wave propagation in real Earth is extraordinarily complex. Therefore, to render the forward simulation computationally manageable within iterative optimization schemes like FWI, geophysicists standardly apply a sequence of physical reductions:

- **Viscoelasticity versus the Non-Attenuating Elastic Assumption:** In the real Earth, rocks are non-perfectly elastic; they suffer from intrinsic energy dissipation (attenuation, quantified by $1/Q$, where Q is the quality factor), where mechanical energy converts into heat, causing high frequencies to damp out over distance. Simulating this requires complex time-convolutions and material memory variables. The first simplification eliminates energy dissipation by assuming a perfectly elastic medium ($Q \rightarrow \infty$, meaning that the attenuation is zero). Since no wave attenuation occurs, the system's energy is conserved and stress depends only on the instantaneous state of deformation.
- **Anisotropy versus the Isotropic Approximation:** Subsurface strata often exhibit directional dependency (*anisotropy*) due to structural micro-layering or aligned fracture networks, meaning that seismic waves travel at different velocities depending on their propagation angle. Assuming that the mechanical properties are invariant with respect to the direction of propagation reduces the multi-parameter anisotropic tensor down to just two spatially varying physical parameters: the Lamé constants, $\lambda(\mathbf{x})$ (incompressibility) and $\mu(\mathbf{x})$ (shear modulus). Within this isotropic elastic framework, longitudinal Compressional waves (*P-waves*) and transverse Shear waves (*S-waves*) become uncoupled, with velocities expressed as:

$$v_p = \sqrt{\frac{\lambda + 2\mu}{\rho}}, \quad v_s = \sqrt{\frac{\mu}{\rho}}.$$

- **Elasticity versus the Acoustic Approximation ($\mu \rightarrow 0$):**

While solid rocks support both P (compressional) and S (shear) wave modes, marine exploration deploys sources and receivers within the water column. While the airguns acting as the seismic source are always towed near the sea surface by the vessel, the receiver placement varies, classifying these experiments into Wide-Angle campaigns (with receivers on the seafloor) and Multichannel campaigns (with receivers floating near the sea surface). In Multichannel setups, since the sensors are floating near the sea surface, it

is physically impossible for them to record shear energy. Consequently, applying the acoustic approximation by setting the subsurface shear modulus to zero ($\mu = 0$, and thus $v_s = 0$) is a valid simplification. This completely removes shear physics from the governing equations, collapsing the complex tensor system into a single, scalar pressure field (p) governed exclusively by acoustic wave propagation.

- **Heterogeneity and the Constant Density Approximation:** Although the subsurface is inherently *heterogeneous*—meaning that physical parameters vary abruptly across geological interfaces and continuously within layers—assuming a homogeneous background density ($\rho = \text{const.}$) simplifies the transport physics. Under this constraint, the coupled system reduces to the classical scalar acoustic wave equation for the pressure or scalar displacement field $u(\mathbf{x}, t)$:

$$\frac{1}{c^2} \frac{\partial^2 u(\mathbf{x}, t)}{\partial t^2} - \nabla^2 u(\mathbf{x}, t) = f(\mathbf{x}, t) , \quad (1.1)$$

where the propagation velocity c is defined strictly through the bulk modulus K (which mathematically replaces λ when $\mu = 0$):

$$c = \sqrt{\frac{K}{\rho}} . \quad (1.2)$$

To numerically solve the acoustic wave equation (1.1), we express it in the canonical mathematical form of a second-order linear PDE,

$$\rho(\mathbf{x}) \frac{\partial^2 u(\mathbf{x}, t)}{\partial t^2} - \nabla \cdot (\mu(\mathbf{x}) \nabla u(\mathbf{x}, t)) = f(\mathbf{x}, t), \quad (1.3)$$

where $u(\mathbf{x}, t)$ represents the scalar wavefield (e.g., the acoustic pressure) at position $\mathbf{x}$ and time t , and $f(\mathbf{x}, t)$ is an external source term.

In this standard mathematical notation, $\rho(\mathbf{x})$ acts as the mass density coefficient and $\mu(\mathbf{x})$ acts as the spatial stiffness coefficient, which are not assumed to be constant throughout the mathematical derivation of the method. Under the previous assumptions, $\mu(\mathbf{x})$ corresponds to the bulk modulus K . Consequently, the local wave propagation velocity can be recovered via the relation $c = \sqrt{\mu/\rho}$.

The equation is defined in the domain $\Omega \times I \subset \mathbb{R}^d \times \mathbb{R}$, where Ω is the medium where the wave propagates, and $I = [0, t_f]$ is the simulated time interval. The problem is completed by assuming known initial conditions $u(\mathbf{x}, 0)$, as well as homogeneous Neumann boundary conditions on $\partial\Omega$, which mathematically represent a perfectly reflecting boundary prior to the implementation of absorbing layers.

1.3 Objectives and Outline of the Thesis

The main objective of this thesis is to develop an efficient and accurate SEM solver for the wave equation in 2-D in the context of FWI to overcome the numerical dispersion problems associated with more traditional FDM solvers. Specifically, we aim to:

- Derive and computationally implement the SEM, detailing the different steps in the method, firstly in 1-D and subsequently in 2-D.
- In 2-D, develop a framework to properly mesh complex geometries to increase the efficiency of the solver.
- Introduce and implement Perfectly Matched Layers to address the artificial truncation of the domain and to be able to simulate wave propagation in real life scenarios.
- Benchmark the performance of the solver and compare it to an FDM implementation to validate its efficiency and accuracy.
- Showcase a real life example (Marmousi model) to prove the flexibility and stability of the solver on a complex geometry.

The outline for the rest of the thesis is as follows. Section 2 contains the mathematical preliminaries necessary to understand this thesis and the SEM formulation. Next, Section 3 implements the SEM in 1-D, showcasing the intuition behind the method that will transition to the 2-D case. Section 4 outlines and implements the SEM in 2-D. Section 5 validates and benchmarks the SEM in 1-D and 2-D, and finally, Section 6 concludes this thesis and provides possible future work directions.

2 Mathematical Preliminaries

In this section we give a brief revision of mathematical results that we will use throughout the thesis. We will mainly use concepts and theory from vector calculus, partial differential equations and numerical methods.

Throughout this section, we set $\Omega \subset \mathbb{R}^n$ to be an open subset with $n \in \mathbb{N}$ fixed, where $\mathbb{R}$ is the set of real numbers and $\mathbb{N}$ the set of natural numbers.

2.1 Vector Calculus

All these classical results and definitions from this subsection can be found in [4] or any other vector calculus introductory book.

Definition 2.1. Let $V \subset \mathbb{R}^m$, and $k \in \mathbb{N}$. We denote $\mathcal{C}^k(\Omega, V)$ the space of functions from Ω to V whose partial derivatives of order up to k exist and are continuous over Ω . If V is not specified, $V = \mathbb{R}$.

Definition 2.2. A *scalar field* f is a mapping from Ω to $\mathbb{R}$,

$$f : \Omega \rightarrow \mathbb{R}.$$

A *vector field* F is a mapping from Ω to $\mathbb{R}^n$,

$$F : \Omega \rightarrow \mathbb{R}^n.$$

We also define the *gradient operator*, which we will use when implementing SEM in higher dimensions.

Definition 2.3. Let $\{e_i\}_{i=1,\dots,n}$ be the standard basis of the cartesian coordinate system $\mathbb{R}^n$ with coordinates $(x_1, \dots, x_n)$. The *gradient* or *nabla* operator ∇ , is a vector operator whose components are the partial derivative operators $\frac{\partial}{\partial x_1}, \dots, \frac{\partial}{\partial x_n}$,

$$\nabla = \sum_{i=1}^n e_i \frac{\partial}{\partial x_i} = \left(\frac{\partial}{\partial x_1}, \dots, \frac{\partial}{\partial x_n} \right).$$

The gradient acts on $f \in \mathcal{C}^1(\Omega)$ scalar fields via scalar multiplication,

$$\nabla f = \sum_{i=1}^n e_i \frac{\partial f}{\partial x_i} = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right),$$

and as a dot product on $F \in \mathcal{C}^1(\Omega, \mathbb{R}^n)$ vector fields,

$$\nabla \cdot F = \left(\frac{\partial}{\partial x_1}, \dots, \frac{\partial}{\partial x_n} \right) \cdot (F_1, \dots, F_n)^t = \sum_{i=1}^n \frac{\partial F_i}{\partial x_i}.$$

The gradient applied to vector fields is sometimes called *divergence* and denoted with the div operator. Notice how ∇ interchanges vector fields and scalar fields.

Proposition 2.4. (*Product rule for divergence*) Let $X \in \mathcal{C}^1(\Omega, \mathbb{R}^n)$ be a vector field and $f \in \mathcal{C}^1(\Omega)$ a scalar field. The divergence of the scalar product $fX \in \mathcal{C}^1(\Omega, \mathbb{R}^n)$ is

$$\nabla \cdot (fX) = \nabla f \cdot X + f \nabla \cdot X.$$

The following is an essential result in vector calculus relating surface and volume integrals known as the *Divergence Theorem*, *Gauss's Theorem* or *Ostrogradsky's Theorem*.

Theorem 2.5. Let $F \in \mathcal{C}^1(\tilde{V}, \mathbb{R}^n)$ be a vector field defined in a neighborhood $\tilde{V}$ of a compact volume $V \subset \mathbb{R}^n$ with a piecewise smooth boundary $\partial V = S$. Then,

$$\int_V \nabla \cdot F \, dV = \int_{\partial V} F \cdot \vec{n} \, dS,$$

where $\vec{n}$ is the normal outward pointing vector to ∂V .

We also state the *Change of variables Theorem*, which we will later use to change the domain of integration. For that we first need to define a *diffeomorphism*.

Definition 2.6. Let $F : U \rightarrow V$ be a map between open subsets of $\mathbb{R}^n$. F is said to be a *diffeomorphism* if it is bijective, and both F, F^{-1} are $\mathcal{C}^1$.

Theorem 2.7. (*Change of Variables*) Let $F : U \rightarrow V$ be a diffeomorphism and let $D^* \subset U$ and $D = F(D^*) \subset V$ be bounded subsets. Then, for a scalar integrable function $f : D \rightarrow \mathbb{R}$,

$$\int_D f(\mathbf{x}) \, dV = \int_{D^*} f(F(\mathbf{x}^*)) |\mathcal{J}(\mathbf{x}^*)| \, dV^*,$$

where $|\mathcal{J}(\mathbf{x}^*)|$ is the absolute value of the determinant of the Jacobian of F .

2.2 Partial Differential Equations

When implementing SEM (and more generally, FEM), we obtain *weak solutions*, which involves transforming PDE's into integral equations via a *weak* approximation. The idea behind the weak form is to relax conditions on the solutions and test functions, specially those related to differentiability and continuity.

To do so, we define *weak derivatives*, which generalize derivatives to integrable functions.

Definition 2.8. Let u be a scalar field which is square integrable on Ω . We say $v : \Omega \rightarrow \mathbb{R}$ is the *weak derivative* of u in the i -th coordinate if for all test functions $\varphi \in \mathcal{C}^\infty(\Omega)$ with compact support contained in Ω it follows,

$$\int_\Omega u \frac{\partial \varphi}{\partial x_i} \, dV = - \int_\Omega v \varphi \, dV.$$

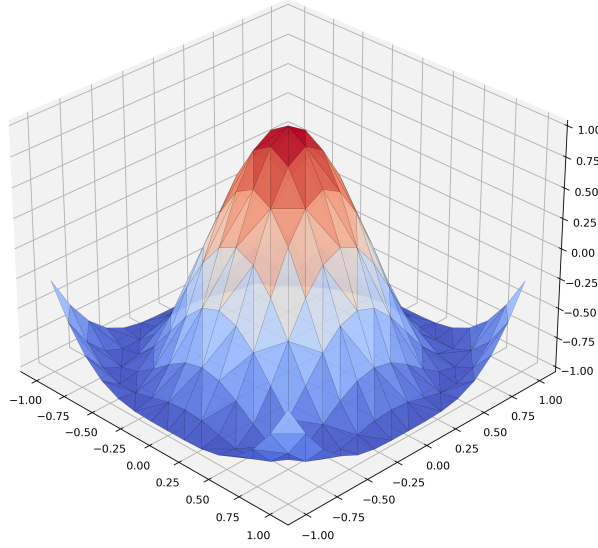


Figure 2: Approximation of a smooth function using FEM.

The weak derivative is motivated by the multivariable integration by parts formula. For functions who are $\mathcal{C}^1$, the i -th derivative is the i -th weak derivative, however, it is not unique as for example, the weak derivative can allow for discontinuities. Moreover, we will denote the i -th weak derivative of u as $\frac{\partial u}{\partial x_i}$.

For functions u not specified to be differentiable that have a weak derivative in all the coordinates we denote the *weak gradient* $\nabla u = \left(\frac{\partial u}{\partial x_1}, \dots, \frac{\partial u}{\partial x_n} \right)$.

Definition 2.9. Let $f : \Omega \rightarrow \mathbb{R}$ be a scalar field. We say f is *square integrable* on Ω if

$$\int_{\Omega} |f(\mathbf{x})|^2 \, dV < \infty.$$

We define the set of square integrable functions on Ω as $L^2(\Omega)$.

Definition 2.10. We denote the *Sobolev space*, $H^1(\Omega)$ to be the space of square integrable functions on Ω that have a weak gradient.

2.3 Finite Element Methods

Numerical solutions to differential equations are often categorized into two frameworks: Finite Difference Methods (FDM) and variational methods, which are commonly implemented via Finite Element Methods (FEM). FDM discretizes both time and space, and replaces the derivatives with finite difference approximations

such as Taylor expansions. Alternatively, variational methods transform the equations into functionals and find functions that minimize/maximize the functional in a specific domain.

While FDM is usually simpler to implement, it is also not as flexible and can be very ineffective in domains with difficult geometries as opposed to variational methods.

FEM initially works locally, as the domain Ω is subdivided into smaller sub-domains, called *elements*. The equation that governs the PDE is transformed and later restricted to each of these elements. *Test functions* are used to find local linear systems of equations that, when put together, form a global system that provide the coefficients to a linear combination of *shape functions*, which approximate the solution u globally.

In FEM, two things are important when applying the method; the way we subdivide the domain and choose our elements, which is called the *mesh*, and the test and shape functions to be used. The basic FEM scheme is the following:

1. Create the mesh by dividing the domain in elements. Elements can have different shapes such as triangles and quadrilaterals.
2. Transform the PDE from its *strong form* to its *weak form*.
3. For each individual element, restrict the equations that govern its behaviour. With these equations and with the use of test functions, generate a local system of equations. The test and shape functions are usually low degree polynomials (linear or quadratic) and can be the same type or not.
4. The local systems are assembled into a global system which provides the coefficients for the global solution. This global system ensures that the forces acting on each element are properly combined so no information is lost.
5. Apply boundary and initial conditions.
6. Solve the system of algebraic equations. It is important what computational solver is chosen to be used, as it can be the difference between an efficient, useful solution and a slow, infeasible one.

Usually, the only way to obtain better precision in FEM is to make the mesh finer (h -refinement), which can come at a high computational cost, as increasing the order of the polynomials (p -refinement) can lead to nonconvergence due to Runge's phenomenon [5].

FEM is one of the most popular methods to solve PDE's and it is widely used in fields such as engineering, mathematical modelling, and physics [6].

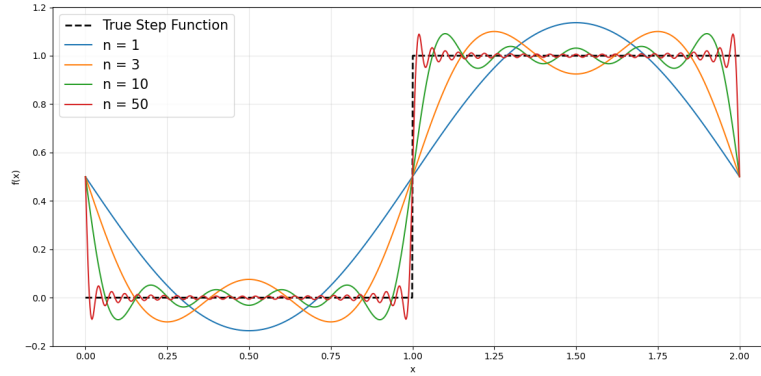


Figure 3: Approximation of a step function using Spectral Methods.

2.4 Spectral Methods

Spectral methods take a global approach. Instead of dividing the domain into smaller parts and finding local functions suited for that specific subdomain, spectral methods compute global functions directly. This can provide better accuracy and can even achieve exponential convergence. However, due to using functions defined in the whole domain, they cannot handle very complex geometries [7].

The main idea spectral methods use is to approximate the solution by a sum of functions that form a basis for a particular space. A prototypical example of this is a Fourier Series, which uses the base of trigonometric functions to approximate periodic functions.

Let s be a periodic function over $[0, L]$, then its Fourier series expansion is

$$s_F(x) = \sum_{n=-\infty}^{\infty} c_n e^{i2\pi \frac{n}{L} x},$$

where the c_n 's are the Fourier coefficients,

$$c_n = \frac{1}{L} \int_0^L s(x) e^{-i2\pi \frac{n}{L} x} dx.$$

If s is square integrable, the series converges in the L^2 sense, whereas if s is piecewise smooth (satisfying Dirichlet's conditions), the series converges pointwise everywhere [8]. Spectral methods use this convergence to truncate the series s_F to obtain an approximation of s . Therefore, increasing accuracy in Spectral Methods consists on increasing the truncation order, which equates to a higher number of terms in the series.

Choosing the basis is not arbitrary, as the computation of the coefficients c_n depend heavily on what basis we use. Other examples of basis functions are polynomials, such as Chebyshev or Legendre polynomials.

2.4 Spectral Methods

In Figure 2 we see an example of a function approximated using FEM, while in Figure 3 we see the spectral method applied to a step function. The two examples highlight the main difference between the two methods: while FEM uses a local approach to approximate the solution in a piecewise manner (by elements), spectral methods use global smooth functions that aim to approximate the solution in the whole domain.

Table 1 summarizes the main differences between FEM, spectral methods and SEM.

Feature	FEM	SM	SEM
Approach	Local	Global	Local
Increasing Resolution	h -refinement	p -refinement	hp -refinement
Typical Shape of Elements	Triangular/ Tetrahedral	-	Quadrilateral/ Hexahedral
Integration Domain	$[0, 1]^k$	-	$[-1, 1]^k$
Basis Functions	Low degree polynomials. Tent (linear) or quadratic at most.	Trigonometric Functions or Polynomials	High degree polynomials. Usually Lagrange or Chebyshev.
Node Distribution	Equidistant/ Uniform	Non-equidistant	Non-equidistant (GLL or Chebyshev points)
Convergence	Polynomial	Exponential	Exponential
Best For	Irregular geometries	Simple Domains	Complex geometries that require high accuracy

Table 1: Comparison of FEM, spectral methods, and SEM numerical methods, where k refers to the dimension of the problem.

3 The Spectral Element Method in 1-D

In this section we describe and implement the SEM in 1 dimension. The outline of the method is mostly shared with how FEM is implemented, with some changes that we highlight.

By implementing first the SEM in 1 dimension we are able to identify the basic ideas behind the method, which will later help translate it to higher dimensions.

SEM aims to combine FEM and spectral methods to obtain the flexibility FEM provides in Ω and the accuracy and rapid convergence of spectral methods.

In SEM, the mesh is divided into elements in the same way as in FEM, but inside each element, instead of using linear or quadratic polynomials to approximate the local shape functions, one uses higher degree polynomials, usually of degree 4 up to 10. However, by using Gauss-Lobatto-Legendre (GLL) points for both interpolation and integration, it keeps the method stable and computationally less expensive.

Using higher degree polynomials provides a better fit for more complex functions and domains while maintaining the number of elements low.

The combination of FEM and spectral methods allows us to increase the accuracy in two different ways (*hp*-refinement): refining the mesh as in FEM (*h*-refinement) or increasing the degree of the polynomials as in a spectral method (*p*-refinement). This added flexibility allows SEM to adapt better to the geometry of the problem while still being computationally efficient and accurate.

However, one of its main disadvantages can be its complex implementation and theoretical difficulty. Applications of the SEM can be seen in seismology, computational fluid dynamics and electromagnetic problems, among others [9].

We start by obtaining the weak formulation of equation (1.3). Next, we describe how to decompose the domain into elements. We then introduce the test and shape functions, along with the interpolation points and GLL quadrature. We follow by computing the elemental mass matrix and highlight the synergy between the test functions, the interpolation points, and the GLL quadrature. We also calculate the stiffness matrix and finally, we assemble the global system and propagate the solution in time. The implementation steps detailed in this section are inspired by and adapted from the methodology outlined in [10].

3.1 Weak Formulation

Equation (1.3) is written in its *strong form*. In this subsection, we will transform it into its *weak form*, which can be seen as an easier «almost» equivalent way of writing the problem. It is «almost» equivalent in the sense that solutions to the strong form produce solutions of the weak form, but the other way around might

not be true. The reasoning behind the weak form is the following: two vectors $u, v \in \mathbb{R}^n$ are equal if and only if, for all $w \in \mathbb{R}^n$ it follows

$$u \cdot w = v \cdot w, \quad (3.1)$$

where $\cdot$ is the euclidean dot product.

Suppose we have a function $F : \mathbb{R}^n \rightarrow \mathbb{R}$ that computes the dot product with an unknown vector $u \in \mathbb{R}^n$ and we wish to find u . Let $B = \{w_i\}_{i=1}^n$ be a basis for $\mathbb{R}^n$ and $\tilde{u}$ such that

$$F(w_i) = \tilde{u} \cdot w_i, \quad \forall w_i \in B,$$

then for any vector $v \in \mathbb{R}^n$, using the linearity of F and the dot product,

$$F(v) = F\left(\sum_{i=1}^n \alpha_i w_i\right) = \sum_{i=1}^n \alpha_i F(w_i) = \sum_{i=1}^n \alpha_i \tilde{u} \cdot w_i = \tilde{u} \cdot v,$$

and using equation (3.1), $\tilde{u} = u$. That is, we would only need to find a u that works for a given basis B , which can be done easily by assembling a linear system of equations for the coefficients of u . Furthermore, if the basis B is orthonormal, we can express u in terms of B , $u = \sum_{i=1}^n \alpha_i w_i$, and find the coefficients α_i by using the orthonormality of the basis,

$$u \cdot w_j = \sum_{i=1}^n \alpha_i w_i \cdot w_j = \alpha_j.$$

With this idea in mind, we take the $L^2(\Omega)$ product for functions and find the solution to (1.3) by generating a system of equations.

Let $v(x) \in H^1(\Omega)$ be a test function. Taking equation (1.3), multiplying by v in both sides and integrating over the domain Ω we obtain

$$\int_{\Omega} v \rho \ddot{u} \, dx - \int_{\Omega} v \nabla \cdot (\mu \nabla u) \, dx = \int_{\Omega} v f \, dx, \quad (3.2)$$

where we used the more classical physics notation $\ddot{u} = \frac{\partial^2 u}{\partial t^2}$ and wrote $\nabla = \frac{\partial}{\partial x}$. This was done to later transition more easily to higher dimensional cases. We can then take equation (3.2) and integrate by parts the second term

$$\int_{\Omega} v \rho \ddot{u} \, dx - \int_{\Gamma} v \mu \nabla u \, dS + \int_{\Omega} \nabla v \mu \nabla u \, dx = \int_{\Omega} v f \, dx,$$

where $\Gamma = \partial\Omega$. As the domain Ω is just a line segment, the boundary is composed by the endpoints $\Gamma = \{a, b\}$, and thus,

$$\int_{\Gamma} v \mu \nabla u \, dS = v \mu \nabla u|_{\Gamma} = v \mu \nabla u|_b - v \mu \nabla u|_a. \quad (3.3)$$

Applying Neumann boundary conditions $\mu \nabla u|_{\Gamma} = 0$, equation (3.3) is equal to 0.

Therefore, if u is a solution to (1.3), for an arbitrary test function $v \in H^1(\Omega)$ the following equation holds:

$$\int_{\Omega} v \rho \ddot{u} \, dx + \int_{\Omega} \nabla v \mu \nabla u \, dx = \int_{\Omega} v f \, dx. \quad (3.4)$$

We call equation (3.4) the weak form of equation (1.3), and u a weak solution if it solves the weak form, in the sense that for any $v \in H^1(\Omega)$, equation (3.4) holds. Notice how the weak form admits solutions with discontinuities in a set of measure 0, while the strong form does not. In fact, as we only impose that $u \in H^1(\Omega)$, u might only be differentiable in the weak sense.

In the context of the previous problem, $F(v) = \int_{\Omega} v f \, dx$, the set of test functions v is the basis B , and the operator is,

$$u \cdot v := \int_{\Omega} v \rho \ddot{u} \, dx + \int_{\Omega} \nabla v \mu \nabla u \, dx.$$

Our objective now is to discretize the weak form (3.4) to numerically solve it. We do this by discretizing the domain Ω and finding an approximation to u that solves the discrete version of the weak form.

3.2 Domain Decomposition

In one dimension, the division of Ω into elements is straightforward if we consider the domain to be an interval $\Omega = [a, b]$. The natural partition and the one we consider is to take $\Omega^i = [x_i, x_{i+1}]$ where the x_i 's are such that

$$a = x_0 < x_1 < \dots < x_N = b.$$

In particular, there are $N + 1$ elements and $\Omega = \cup_e \Omega^e$.

The points $\{x_i\}_{i=0, \dots, N}$ are called *nodes*. How we choose the x_i 's is important as different geometries in the domain can lead to different numerical results and slower/faster rates of convergence. Ideally, in regions where the solution u is near constant, the points x_i should be spaced far apart, while in regions where u is rapidly changing, the nodes should be close together.

The division of the domain in elements is done to later integrate each Ω^e individually. For reasons we will state in the next Subsection 3.3, we will map each element to $\Lambda = [-1, 1]$. In FEM, this mapping is usually done to the unit interval $[0, 1]$, however, in SEM, in order to take advantage of GLL quadrature we must map into Λ .

The function $\mathcal{F}_e$ that maps Λ to Ω^e is

$$\begin{aligned}\mathcal{F}_e: \Lambda &\rightarrow \Omega^e \\ \xi &\mapsto h_e \frac{\xi + 1}{2} + x_e,\end{aligned}$$

where $h_e = x_{e+1} - x_e$. In the case where all elements are of the same length, $h_e = \frac{b-a}{N}$.

As $\mathcal{F}_e$ is affine and invertible, it is a diffeomorphism, which we will later use to change variables.

3.3 GLL Quadrature

When integrating the weak form, we first map each element Ω^e to Λ and change the integration domain. We do this to integrate using GLL quadrature, which transforms an integral in the interval Λ into a finite weighted sum. In particular for a function h ,

$$\int_{-1}^1 h(x) \, dx \simeq \sum_{i=0}^p \omega_i h(\xi_i),$$

where ω_i are the weights obtained by the formula

$$\omega_i = \frac{2}{p(p+1)[P_p(\xi_i)^2]},$$

and the collocation points or GLL points ξ_i , are the roots of the first derivative of the Legendre polynomial of degree p ,

$$P_p(\xi) = \frac{1}{2^p p!} \frac{d^p}{d\xi^p} (\xi^2 - 1)^p,$$

plus the endpoints $\xi = -1, 1$. Figure 4 showcases the first 6 Legendre polynomials together with the GLL points.

An important distinction between Gauss quadrature and GLL quadrature is that in GLL quadrature, the endpoints are always collocation points ξ_i . Consequently, the quadrature is exact up to polynomials of degree $2p - 1$ instead of $2p + 1$. We will later see that using specifically the ξ_i 's as collocation points will make the mass matrix diagonal, which will simplify calculations. These results and an extended explanation of Gauss quadrature and Legendre polynomials can be found on [7].

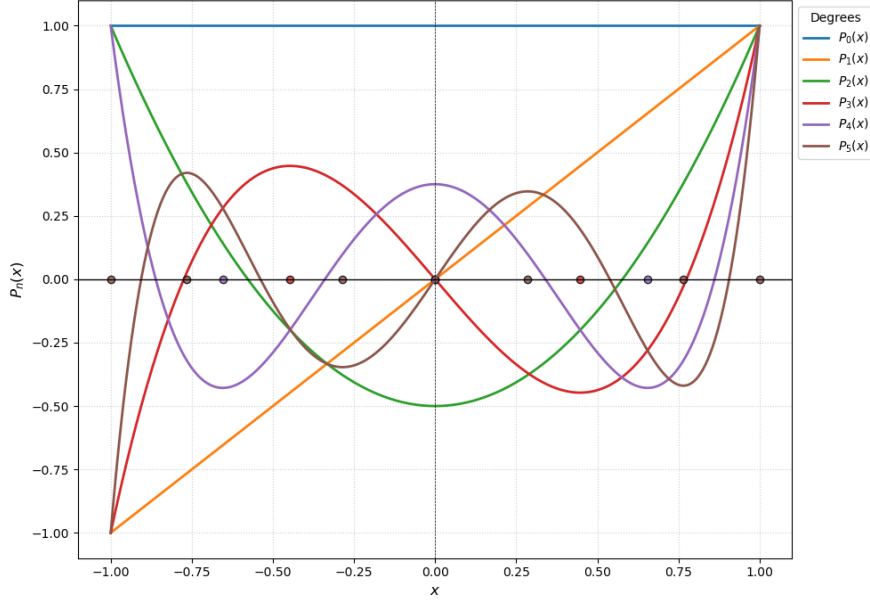


Figure 4: First 6 Legendre polynomials and interior GLL nodes, corresponding to $P'_n(x) = 0$ and marked in the x -axis.

3.4 Local Interpolation and Shape Functions

With the domain Ω divided into elements, we can now describe how to approximate the solution u . As in FEM, this is done by approximating u inside each element by a linear combination of shape functions and later taking the sum of all the local functions,

$$u(x, t) \simeq \sum_{e=0}^N \tilde{u}_e(x, t),$$

where $\tilde{u}_e$ is the function approximated in element e . The approximation $\tilde{u}_e$ has support only in Ω^e , as to not change the value of $\tilde{u}_{e'}$ in other elements. Nevertheless, as elements share nodes, we will have to consider their interactions, in particular at the endpoints of Ω^e . From this point forward, as to not crowd the notation, we will not write the tilde, understanding that we are calculating a numerical approximation and not u .

Instead of finding u_e , we define $u^e(\xi, t) := u_e(\mathcal{F}_e(\xi), t)$ in order to integrate in Λ . To find u^e , we consider the shape functions to be the Lagrange polynomials ℓ_i in the GLL points ξ_i , which satisfy the Kronecker delta property δ_{ij} ,

$$\ell_i(\xi_j) = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j, \end{cases}$$

and constitute a basis for the space of polynomials of degree less or equal than p . Taking u^e as a linear combination of them,

$$u^e(\xi, t) = \sum_{i=0}^p u_i^e(t) \ell_i(\xi). \quad (3.5)$$

The final objective of SEM is to find the values of the displacement u on the nodes, $\{u_i^e(t)\}_{i=0,\dots,p}$ for each element e , as they represent the approximation to u in the interpolation points.

If we have p collocation points, the Lagrange polynomials are

$$\ell_i(\xi) = \prod_{j \neq i}^p \frac{\xi - \xi_j}{\xi_i - \xi_j}$$

In general, in FEM only interpolating polynomials of degree 1 (tent functions) or 2 (quadratic) are used, but in SEM a higher degree is required, usually $p = 4, \dots, 10$. Using a low degree loses the advantages SEM has over FEM, while using a very high degree makes the method computationally infeasible [11].

3.5 Integration of the Weak Form

With all the approximations explained, we are ready to integrate the weak form for each element, transforming (3.4) into a linear system where the variables are $\{\{u_i^e(t)\}_{i=0,\dots,p}\}_{e=0,\dots,N}$. As noted in Subsection 3.4, elements share the edge nodes, which will make the local submatrices overlap in the global assembly. This fact is important and will be explained in further detail in Subsection 3.6.

3.5.1 Elemental Mass Matrix

For a fixed $t \in I$, integrating in Ω^e and via the change of variables by applying $\mathcal{F}_e$, the first term of equation (3.4) becomes

$$\int_{\Omega^e} v(x) \rho(x) \ddot{u}(x, t) \, dx = \int_{\Lambda} v^e(\xi) \rho^e(\xi) \ddot{u}^e(\xi, t) |\mathcal{J}^e| \, d\xi,$$

where we defined $\rho^e, v^e, \ddot{u}^e$ analogously to u^e . The term $\mathcal{J}^e$ is the Jacobian of the change of variables, $\mathcal{J}^e = \frac{dx}{d\xi}$. Using the approximation by a combination of shape functions (3.5) in v^e and $\ddot{u}^e$,

$$\int_{\Lambda} v^e(\xi) \rho^e(\xi) \ddot{u}^e(\xi, t) |\mathcal{J}^e| \, d\xi \simeq \int_{\Lambda} \rho^e \left(\sum_{i=0}^p v_i^e \ell_i(\xi) \sum_{j=0}^p \ddot{u}_j^e \ell_j(\xi) \right) |\mathcal{J}^e| \, d\xi,$$

where we omitted the time t in $\ddot{u}_j^e(t)$. Applying GLL quadrature,

$$\int_{\Lambda} \rho^e \left(\sum_{i=0}^p v_i^e \ell_i(\xi) \sum_{j=0}^p \ddot{u}_j^e \ell_j(\xi) \right) |\mathcal{J}^e| d\xi \simeq \sum_{k=0}^p \rho^e(\xi_k) \omega_k \left(\sum_{i=0}^p v_i^e \ell_i(\xi_k) \sum_{j=0}^p \ddot{u}_j^e \ell_j(\xi_k) \right) |\mathcal{J}^e(\xi_k)|$$

defining $\rho_k^e := \rho^e(\xi_k)$ and analogously for $v^e, \ddot{u}^e, \mathcal{J}^e$, and $\ell_j(\xi_k) = \delta_{jk}$, we obtain

$$\sum_{k=0}^p \rho^e(\xi_k) \omega_k \left(\sum_{i=0}^p v_i^e \ell_i(\xi_k) \sum_{j=0}^p \ddot{u}_j^e \ell_j(\xi_k) \right) |\mathcal{J}^e(\xi_k)| = \rho_i^e \omega_i \ddot{u}_i^e |\mathcal{J}_i^e|,$$

where we set the test function to be $v^e = \ell_i$ for an $i \in \{0, \dots, p\}$. Finally, let us define the *mass matrix* for element e as the matrix with entries $M_{ij}^e = \rho_i^e \omega_i |\mathcal{J}_i^e| \delta_{ij}$. Because M^e is diagonal, the sum collapses to a single term

$$\rho_i^e \omega_i \ddot{u}_i^e |\mathcal{J}_i^e| = \sum_{j=0}^p M_{ij}^e \ddot{u}_j^e,$$

which will later prove useful when solving the final linear system.

Notice how M^e being diagonal is not a coincidence: choosing the GLL points for both the quadrature and the interpolation made it so the Lagrange polynomials behave as Kronecker deltas on the entries of M^e . This will save many computations as the global mass matrix M will have to be inverted, which is trivially done in diagonal matrices.

3.5.2 Elemental Stiffness Matrix

The *elemental stiffness matrix* is the matrix associated with the displacement terms obtained from the weak form.

By the chain rule, when performing the change of variable we have to transform the operator $\nabla_x = \frac{\partial}{\partial x} = \frac{\partial \xi}{\partial x} \frac{\partial}{\partial \xi}$, so the second term in equation (3.4) is

$$\begin{aligned} \int_{\Omega^e} \nabla_x v \mu \nabla_x u dx &= \int_{\Lambda} \nabla_x v^e(\xi) \mu^e(\xi) \nabla_x u^e(\xi, t) |\mathcal{J}^e(\xi)| d\xi \\ &= \int_{\Lambda} \frac{\partial \xi}{\partial x} \nabla_{\xi} v^e(\xi) \mu^e(\xi) \frac{\partial \xi}{\partial x} \nabla_{\xi} u^e(\xi, t) |\mathcal{J}^e(\xi)| d\xi \\ &= \int_{\Lambda} \left(\frac{\partial \xi}{\partial x} \right)^2 \nabla_{\xi} v^e(\xi) \mu^e(\xi) \nabla_{\xi} u^e(\xi, t) |\mathcal{J}^e(\xi)| d\xi. \end{aligned}$$

Now, by applying the same approximations and same test function as before,

$$\begin{aligned}
 & \simeq \int_{\Lambda} \left(\frac{\partial \xi}{\partial x} \right)^2 \left(\sum_{i=0}^p v_i^e \ell'_i(\xi) \right) \mu^e(\xi) \left(\sum_{j=0}^p u_j^e \ell'_j(\xi) \right) |\mathcal{J}^e(\xi)| \, d\xi \\
 & \simeq \sum_{k=0}^p \left(\omega_k \left(\frac{\partial \xi}{\partial x}(\xi_k) \right)^2 \ell'_i(\xi_k) \mu^e(\xi_k) \left(\sum_{j=0}^p u_j^e \ell'_j(\xi_k) \right) |\mathcal{J}^e(\xi_k)| \right) \\
 & = \sum_{j=0}^p \left(u_j^e \sum_{k=0}^p \mu_k^e \omega_k \left(\frac{\partial \xi}{\partial x}(\xi_k) \right)^2 |\mathcal{J}_k^e| \ell'_{ik} \ell'_{jk} \right) \\
 & = \sum_{j=0}^p u_j^e K_{ij}^e
 \end{aligned}$$

where we defined $K_{ij}^e = \sum_{k=0}^p \mu_k^e \omega_k \left(\frac{\partial \xi}{\partial x}(\xi_k) \right)^2 |\mathcal{J}_k^e| \ell'_{ik} \ell'_{jk}$ and $\ell'_{ik} = \ell'_i(\xi_k)$. Notice how K^e is not diagonal.

3.6 Assembly of the Global System

With the elemental mass and stiffness matrices defined, we form the global linear system of equations. In FEM this process is usually denoted as the *assembly* of the system.

Fixed a time $t \in I$, for an element e and the test function $\ell_i(\xi)$, the weak form restricted to Ω^e approximates to

$$\sum_{j=0}^p M_{ij}^e \ddot{u}_j^e + \sum_{j=0}^p u_j^e K_{ij}^e = \int_{\Lambda} \ell_i(\xi) f_t^e(\xi) |\mathcal{J}^e(\xi)| \, d\xi. \quad (3.6)$$

By applying GLL quadrature to the right hand side of (3.6), we obtain

$$\sum_{j=0}^p M_{ij}^e \ddot{u}_j^e + \sum_{j=0}^p u_j^e K_{ij}^e = f_t^e(\xi_i) |\mathcal{J}_i^e| \omega_i.$$

Doing this for $i = 0, \dots, p$ and writing it in matrix form,

$$M^e \ddot{\mathbf{u}}_t^e + K^e \mathbf{u}_t^e = \mathbf{f}_t^e, \quad (3.7)$$

where we set the vector $\mathbf{f}_t^e = (f_t^e(\xi_i) |\mathcal{J}_i^e| \omega_i)_{i=0, \dots, p}$.

Recalling that the elements formed a partition of the domain, the weak form can be written as

$$\sum_e \left(\int_{\Omega^e} v \rho \ddot{u} \, dx + \int_{\Omega^e} \nabla v \mu \nabla u \, dx \right) = \sum_e \int_{\Omega^e} v f \, dx. \quad (3.8)$$

By combining equations (3.7) and (3.8) we assemble the global system

$$\sum_e (M^e \ddot{u}_t^e + K^e u_t^e) = \sum_e \mathbf{f}_t^e.$$

Therefore, by defining the *global mass and stiffness matrices* as the sum of the elemental mass and stiffness matrices, we can write the previous equation as,

$$M \ddot{u}_t + K u_t = \mathbf{f}_t. \quad (3.9)$$

An abuse of notation was done when «summing» the local matrices, as they have size $(p+1) \times (p+1)$ and the sum would result in a matrix of the same size. This sum is performed globally according to global and not local indices.

A node has two indices: the local index used to locate it inside its corresponding element, and a global one used for the global system. While the translation between local and global indices is trivial in one dimension, it becomes a challenge in higher dimensions and we will have to introduce the *connectivity matrix*, explained in Subsection 4.4.

A node in element e with local index i has global index $e \times p + i$, and thus the coefficient (i, j) of the elemental mass or stiffness matrix of element e becomes $(e \times p + i, e \times p + j)$. For example, if we have 2 elements with $p = 2$, the global stiffness matrix is

$$K = \begin{pmatrix} K_{0,0}^0 & K_{0,1}^0 & K_{0,2}^0 & 0 & 0 \\ K_{1,0}^0 & K_{1,1}^0 & K_{1,2}^0 & 0 & 0 \\ K_{2,0}^0 & K_{2,1}^0 & (K_{2,2}^0 + K_{0,0}^1) & K_{0,1}^1 & K_{0,2}^1 \\ 0 & 0 & K_{1,0}^1 & K_{1,1}^1 & K_{1,2}^1 \\ 0 & 0 & K_{2,0}^1 & K_{2,1}^1 & K_{2,2}^1 \end{pmatrix},$$

where we see that the node of global index 2 is shared between element 0 and element 1.

Finally, using an explicit central difference scheme for $\ddot{u}$, we can rewrite equation (3.9) as

$$u_{t_{n+1}} = \Delta t^2 [M^{-1}(\mathbf{f}_{t_n} - K u_{t_n})] + 2u_{t_n} - u_{t_{n-1}}. \quad (3.10)$$

Given the initial conditions, we can recursively calculate the u_{t_n} 's, propagating the solution forward in time to obtain the approximate shape of u for all $t_n \in I$.

4 The Spectral Element Method in 2-D

In this section, we describe and implement the SEM in 2 dimensions. The idea behind the method is the same; however, there are some specific details that add a layer of complexity which we will have to address.

Recalling the wave equation (1.3), we write it out in 2-D,

$$\rho(x, y) \frac{\partial^2 u(x, y, t)}{\partial t^2} - \nabla \cdot (\mu(x, y) \nabla u(x, y, t)) = f(x, y, t), \quad (4.1)$$

where now the nabla operator is $\nabla = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y} \right)$ and $u : \Omega \times I \rightarrow \mathbb{R}$ is a scalar function.

4.1 Weak Formulation

To derive the weak formulation of the wave equation in 2-D—and for n dimensions for that matter—we will make use of the higher order versions of the theorems that we applied to derive the weak form in 1-D.

After multiplying by a test function $v : \Omega \rightarrow \mathbb{R}$, $v \in H^1(\Omega)$, and integrating both sides of equation (4.1), we obtain

$$\int_{\Omega} v \rho \ddot{u} \, dA - \int_{\Omega} v \nabla \cdot (\mu \nabla u) \, dA = \int_{\Omega} v f \, dA, \quad (4.2)$$

to which we apply the vector product rule 2.4,

$$v \nabla \cdot (\mu \nabla u) = \nabla \cdot (v \mu \nabla u) - \mu \nabla v \cdot \nabla u,$$

and substituting it into the second term of (4.2),

$$\int_{\Omega} v \rho \ddot{u} \, dA - \left(\int_{\Omega} \nabla \cdot (v \mu \nabla u) \, dA - \int_{\Omega} \mu \nabla v \cdot \nabla u \, dA \right) = \int_{\Omega} v f \, dA. \quad (4.3)$$

Recalling the Divergence Theorem 2.5 and applying it to the second term of the left hand side,

$$\int_{\Omega} \nabla \cdot (v \mu \nabla u) \, dA = \int_{\partial\Omega} v \mu \nabla u \cdot \vec{n} \, dS,$$

where $\vec{n}$ is the normal vector to $\partial\Omega$ pointing outwards. This term becomes 0 due to Neumann boundary conditions, $\mu \nabla u \cdot \vec{n}|_{\partial\Omega} = 0$.

Therefore, the weak formulation reads as

$$\int_{\Omega} v \rho \ddot{u} \, dA + \int_{\Omega} \mu \nabla v \cdot \nabla u \, dA = \int_{\Omega} v f \, dA. \quad (4.4)$$

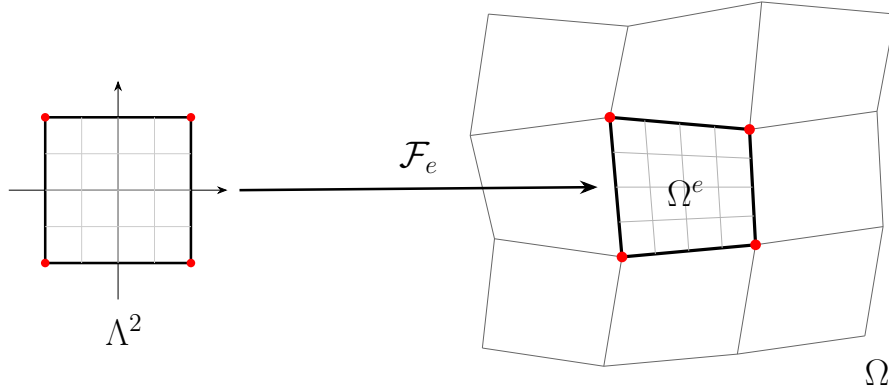


Figure 5: Representation of the transformation $\mathcal{F}_e : \Lambda^2 \rightarrow \Omega^e$ with 4 anchor points.

4.2 Domain Decomposition

We will implement the SEM with quadrilateral elements. While this makes it less apt for complex geometries compared to triangular implementations, using quadrilaterals makes the method less computationally expensive and easier to implement [12]. For this reason, the most common way to use SEM is via quadrilateral elements as opposed to triangular.

When integrating the weak form in each element Ω^e , we will map them to $\Lambda^2 = [-1, 1]^2$. We define again $\mathcal{F}_e$ the mapping from Λ^2 to Ω^e ,

$$\mathcal{F}_e : \Lambda^2 \rightarrow \Omega^e$$

$$(\xi, \eta) \mapsto \sum_{i=1}^{Q+1} N_i(\xi, \eta) \mathbf{x}_i^e.$$

The points $\mathbf{x}_i^e \in \Omega^e$ are the anchor points and will act as references to map points in Λ^2 to points in Ω^e , while Q is the degree of the mapping.

The simplest case to consider is for the Ω^e to be straight edged quadrilaterals, represented in Figure 5. In this scenario, we map the vertices w_j of the square Λ^2 to the vertices $\mathbf{x}_i^e$ of Ω^e . To achieve this, the functions N_i are a product of Lagrange polynomials of degree 1 such that $N_i(w_j) = \delta_{ij}$. These can easily be computed to be

$$N_i(\xi, \eta) = \ell_j(\xi) \ell_k(\eta),$$

where the ℓ_i are the Lagrange polynomials of degree 1.

For more complex geometries (such as curved boundaries), we must increase the number of anchor points and the degree of the mapping functions N_i . For a mapping of degree Q , there are $(Q + 1)^2$ anchor points per element. For instance, a quadratic mapping ($Q = 2$) requires 9 anchor points.

Using quadrilateral elements differs from the more traditional usage of triangular elements in FEM, and is one of the main distinctions between the two methods.

4.3 Local Interpolation, Shape Functions and GLL Quadrature

As in 1-D, we approximate the functions inside each element by taking a linear combination of the shape functions. The higher dimension shape functions are the product of the Lagrange polynomials on the GLL points. In the 2-D case,

$$u^e(\xi, \eta, t) \simeq \sum_{i,j=0}^p u_{ij}^e(t) \ell_i(\xi) \ell_j(\eta). \quad (4.5)$$

Once again, we will integrate the weak form in Λ^2 using GLL quadrature,

$$\int_{\Lambda^2} h(\xi, \eta) dA \simeq \sum_{i,j=0}^p \omega_i \omega_j h(\xi_i, \eta_j),$$

for a function h , which in combination with (4.5) will make for an easier computation for M .

4.4 Mapping of Local indices to Global indices

Thus far, changing the problem from 1-D to 2-D has been straightforward by just adding one extra dimension where it was required. However, if we recall, the weak formulation in the 1-D case resulted in a system of linear equations which were used to propagate the solution forward in time by finding approximations to points of u , $\{u_i\}_{i=0,\dots,T_{\text{nodes}}}$, where T_{nodes} is the total number of points in the global system. This system was assembled by computing the different pieces (elements) first locally and then summing them up by looking at the position of each node globally.

In 2 dimensions, the indexing of the nodes is not as simple, as there is no «canonical» order. Moreover, nodes can now be shared by more than two elements. This makes the implementation specially complex and the transformation from local to global indexing non-trivial. As this transformation is done every time we propagate the solution in time, we need a quick way to do it.

Therefore, in order to translate local indices into global ones, we define a map $\mathcal{M}$ that takes the local index of a node inside an element and outputs its global index,

$$\begin{aligned} \mathcal{M}: \{0, \dots, p\}^2 \times \{0, \dots, N_e\} &\rightarrow \{0, \dots, T_{\text{nodes}}\} \\ (i, j, e) &\mapsto \mathcal{M}(i, j, e). \end{aligned}$$

With this notation, we say two elements e_l, e_k share a node with local index (i_l, j_l) (resp. (i_k, j_k)) if $\mathcal{M}(i_l, j_l, e_l) = \mathcal{M}(i_k, j_k, e_k)$.

To compute this mapping we define the connectivity matrix C , which will transform local coordinates (i, j, e) to global coordinates. The column index e of C indicates the element index while the row $n = i(p + 1) + j$ indicates the local index (i, j) of the node. Thus $\mathcal{M}(i, j, e) = C_{i(p+1)+j, e}$.

For example, in the following grid with 4 elements and 4 nodes per element (2 nodes per direction per element, $p = 1$):

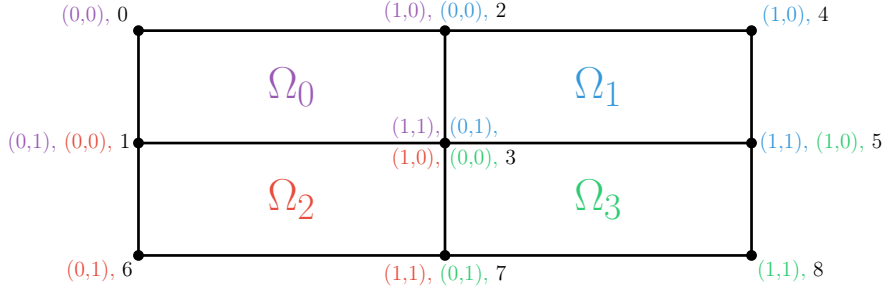


Figure 6: Global grid of a SEM example. Each element has a color assigned, and each node has a local index (colored according to which element it belongs to) and a global index (in black).

The connectivity matrix is

$$C = \begin{pmatrix} 0 & 2 & 1 & 3 \\ 1 & 3 & 6 & 7 \\ 2 & 4 & 3 & 5 \\ 3 & 5 & 7 & 8 \end{pmatrix}.$$

With C calculated, if for instance we need to access the global index of the node in element 1 with local index $(1, 1)$, we look at column 1 and row $1 \times 2 + 1 = 3$, $C_{3,1} = 5$, recalling that the numbering of rows and columns start with 0.

4.5 Integration of the Weak Form

To obtain the elemental mass and stiffness matrices we do as before and integrate the weak form using the approximations we have described. The indices (i, j) and (m, n) , when looked inside the local matrices, correspond to $i(p + 1) + j$ and $m(p + 1) + n$ respectively, as described in the previous Subsection 4.4.

Throughout this whole subsection we fix the time $t \in I$.

4.5.1 Elemental Mass Matrix

The process and approximations used to derive the elemental mass matrix are the same as in the 1-D case, however, having in mind now that the Jacobian is a 2 by 2 matrix, of which we take the absolute value of the determinant,

$$|\mathcal{J}^e(\xi, \eta)| = \left| \begin{array}{cc} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{array} \right|_{(\xi, \eta)}.$$

Hence, by applying GLL quadrature, the approximation of $\ddot{u}$ as in equation (4.5), and the same process as in 3.5.1, for each element e ,

$$\int_{\Omega^e} v \rho \ddot{u} \, dA \simeq \ddot{u}_{ij}^e M_{ij}^e$$

for the test function $v(\xi, \eta) = \ell_i(\xi)\ell_j(\eta)$ for $i, j = 0, \dots, p$, where we denoted

$$M_{ij}^e = \omega_i \omega_j \rho_{ij}^e |\mathcal{J}_{ij}^e|,$$

the coefficients for the elemental mass matrix M^e .

4.5.2 Elemental Stiffness Matrix

In the 1-D approximation for the coefficients of K , we first had to apply the chain rule to the operator $\nabla_{(x,y)}$. We recall the chain rule for the multivariable case,

$$\nabla_{(\xi, \eta)} v^e = \nabla_{(x, y)} v \cdot \begin{pmatrix} \frac{\partial \xi}{\partial x} & \frac{\partial \xi}{\partial y} \\ \frac{\partial \eta}{\partial x} & \frac{\partial \eta}{\partial y} \end{pmatrix} = \nabla_{(x, y)} v \cdot (\mathcal{J}^e)^{-1}.$$

The Jacobian $\mathcal{J}^e$ is invertible at every point, as $\mathcal{F}_e$ is a diffeomorphism, which allows us to write $(\mathcal{J}^e)^{-1}$. Thus,

$$\nabla_{(x, y)} v \cdot \nabla_{(x, y)} u = \nabla_{(\xi, \eta)} v^e ((\mathcal{J}^e)^{-1} (\mathcal{J}^e)^{-T}) (\nabla_{(\xi, \eta)} u^e)^T,$$

and substituting it into the second term of the weak formulation (4.4), we obtain

$$\int_{\Omega^e} \mu \nabla v \cdot \nabla u \, dA = \int_{\Lambda^2} \mu^e \nabla_{(\xi, \eta)} v^e ((\mathcal{J}^e)^{-1} (\mathcal{J}^e)^{-T}) (\nabla_{(\xi, \eta)} u^e)^T |\mathcal{J}^e| \, dA.$$

Multiplying the term $(\mathcal{J}^e)^{-1} (\mathcal{J}^e)^{-T}$

$$(\mathcal{J}^e)^{-1} (\mathcal{J}^e)^{-T} = \begin{pmatrix} \frac{\partial \xi}{\partial x} & \frac{\partial \xi}{\partial y} \\ \frac{\partial \eta}{\partial x} & \frac{\partial \eta}{\partial y} \end{pmatrix} \begin{pmatrix} \frac{\partial \xi}{\partial x} & \frac{\partial \xi}{\partial y} \\ \frac{\partial \eta}{\partial x} & \frac{\partial \eta}{\partial y} \end{pmatrix}^T = \begin{pmatrix} G_{11} & G_{12} \\ G_{12} & G_{22} \end{pmatrix},$$

where the G_{ij} are the coefficients for the metric tensor,

$$\begin{cases} G_{11} = \left(\frac{\partial \xi}{\partial x}\right)^2 + \left(\frac{\partial \xi}{\partial y}\right)^2 \\ G_{12} = \frac{\partial \xi}{\partial x} \frac{\partial \eta}{\partial x} + \frac{\partial \xi}{\partial y} \frac{\partial \eta}{\partial y} \\ G_{22} = \left(\frac{\partial \eta}{\partial x}\right)^2 + \left(\frac{\partial \eta}{\partial y}\right)^2. \end{cases}$$

By applying the operator $\nabla_{(\xi, \eta)}$ to the approximation $u^e(\xi, \eta, t) \simeq \sum_{ij} u_{ij}^e \ell_i(\xi) \ell_j(\eta)$ and to v , and by integrating using GLL quadrature, we obtain the final expression for the integral,

$$\int_{\Omega^e} \mu \nabla v \cdot \nabla u \, dA \simeq \sum_{m,n=0}^p u_{mn}^e K_{mn}^{ij,e},$$

where the coefficients of K^e are

$$K_{mn}^{ij,e} = \sum_{l,k=0}^p \omega_l \omega_k \mu_{lk}^e |\mathcal{J}_{lk}^e| T_{mn}^{ij,e}(\xi_l, \eta_k),$$

where we defined the $T_{mn}^{ij,e}$ as

$$\begin{aligned} T_{mn}^{ij,e}(\xi, \eta) = & G_{11}(\ell'_i(\xi) \ell_j(\eta) \ell'_m(\xi) \ell_n(\eta)) + \\ & G_{12}(\ell'_i(\xi) \ell_j(\eta) \ell_m(\xi) \ell'_n(\eta) + \ell_i(\xi) \ell'_j(\eta) \ell'_m(\xi) \ell_n(\eta)) + \\ & G_{22}(\ell_i(\xi) \ell'_j(\eta) \ell_m(\xi) \ell'_n(\eta)). \end{aligned}$$

Note that the subindex (m, n) refers to the variables u_{mn}^e , the (i, j) refers to the test functions $v(\xi, \eta) = \ell_i(\xi) \ell_j(\eta)$ and the (l, k) refers to the functions evaluated at point (ξ_l, η_k) .

4.6 Mesh Algorithms

We implemented the SEM solver in two different ways regarding the domain Ω , one that reads a `.dat` file containing the node information (`MESH_MODE=1`) and another one where we manually input the parameters for a rectangular grid (`MESH_MODE=0`). The pseudocode for the two different initializations of the mesh can be found in Appendix A.1.

For the rectangular grid, the parameters were the number of elements per direction, n_x and n_y and the degree of the polynomials used in the GLL quadrature p , which equates to $p+1$ nodes per direction per element. Therefore, the number of elements is $N_e = n_x n_y$ and the number of nodes per element $N_n = (p+1)^2$.

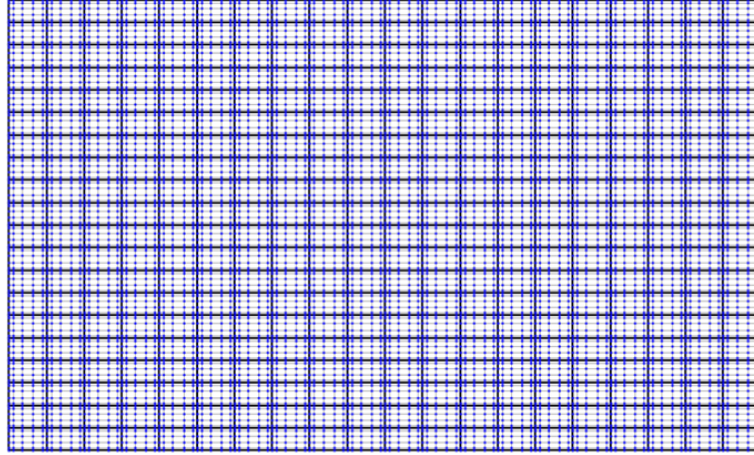


Figure 7: Rectangular mesh example with 400 elements and $p = 6$ initialized with `MESH_MODE=0`.

In this case, to compute the connectivity matrix and the order of the points in the grid, we did as in Figure 6: elements were numbered from left to right and up to down and nodes inside each element were numbered from up to down and left to right. The idea behind the computation of C was to first define an auxiliary matrix A initiated to 0 of the same size as the number of nodes per direction $A \in M_{n_y p+1, n_x p+1}(\mathbb{N})$ to «embed» the elements by order. For each node inside each element, if the position of the node was filled with an index $n \neq 0$, the node had global index n , if not, the entry was filled with a number updated by a counter. This way, all local indices have a unique global index and all global nodes have at least one corresponding local node.

This was done at the same time as we computed the coordinates (x, y) of the nodes, so the (x, y) coordinates of $\mathcal{F}_e(\xi_i, \eta_j)$ matched those of the node with global index $\mathcal{M}(i, j, e)$. An example of a mesh initialized in this way can be seen in Figure 7.

The code also accepts `.dat` files that contain the following information in this order:

- Number of elements and number of nodes.
- Coordinates of the nodes ordered by their global indices.
- The connectivity matrix read line by line.
- The number of nodes in the boundary $\partial\Omega$.
- A list containing the global index of the nodes in the boundary.

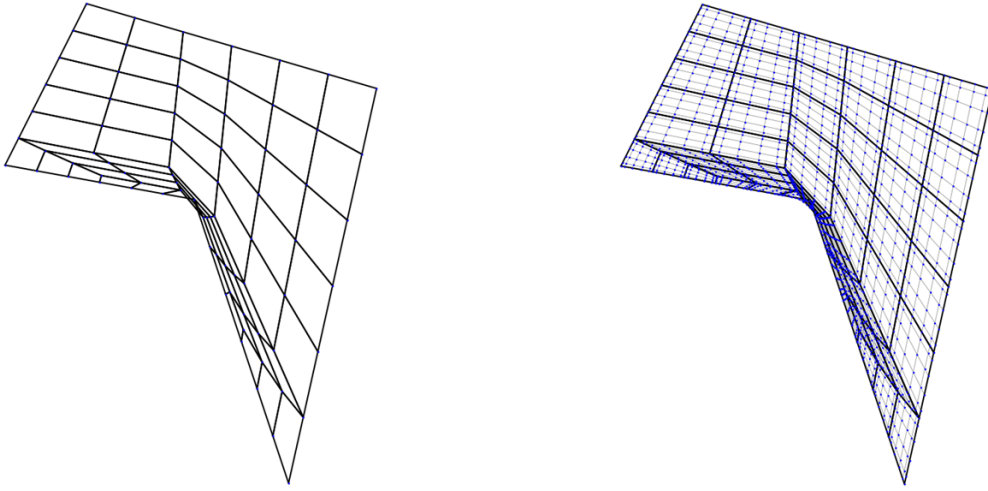


Figure 8: On the left, mesh created by SALOME, on the right, transformed and adapted to use in a SEM simulation using `MESH_MODE=1`.

To create the mesh we used the SALOME software (Version 9.15.0) [13] with the SMESH module. SALOME allows us to create and mesh complex geometries. For this reason, it is widely used in numerical simulations, data analysis and CAD, among others.

Although SALOME is used in many FEM applications, it does not have the option to mesh using GLL points, which is necessary in SEM. To create a mesh useful for our simulation, we programmed a mesh converter in C that takes as input a mesh generated by SALOME and uses it as a base to create a higher order mesh with the GLL points.

The mesh created with SALOME was done with the Quadrangle: Mapping meshing algorithm, which meshes a geometry using only quadrilaterals, which constituted the elements Ω^e . The information of the mesh contained the coordinates for the vertices of the elements, the global numbering of the vertices and connectivity matrix, and which vertices were on the boundary of the domain.

Since one cannot assume any structure in the mesh, we could not compute the connectivity matrix as we did for the manual mesh. Instead, after the mesh converter reads the information outputted by SALOME, it iterates through all the elements to create the GLL points while numbering them locally and globally. To check if an edge node in an element was already numbered globally, we checked if the coordinates were already «explored» by looking at the list of coordinates of the previously numbered edge nodes.

To apply the boundary conditions, we also had to identify all the nodes on the boundary. As SALOME already indicates which vertices are on the boundary, to check if a node is on the boundary, we did so by verifying if the node was on any of the lines defined by the boundary vertices. For a pair of vertices v_i, v_j , and a node w , we first computed the determinant between $\overrightarrow{v_i v_j}$ and $\overrightarrow{v_i w}$. If the determinant was not 0, w could not lie in the line defined by v_i and v_j . If it was 0, we checked if w was inside the bounding box generated by v_i and v_j , if so, the node was assigned to be a boundary node.

On Figure 8 we see an example of a non trivial geometry created by SALOME and converted to a mesh for our SEM simulation. While it is unlikely that we ever use the solver on geometries like that one, in real life simulations rectangular geometries will have non constant density and bulk modulus. In these cases, it is interesting to adapt the mesh to accommodate for the properties inside each region. Non constant sizes for the elements are needed to run the simulation faster without compromising accuracy, as taking the smallest size needed for a region as a constant across the whole domain is unnecessary and time consuming for regions that do not need such small elements.

In Section 5 we see a real life example of this behaviour in the Marmousi model.

4.7 Perfectly Matched Layers

When defining the problem, we initially chose the boundary condition of the wave equation to be of homogeneous Neumann type. This was done to simplify the manipulation of the equation and avoid calculating a line integral on the boundary Γ .

In realistic wave propagation simulations for FWI, the domain Ω can effectively be considered infinite, as the wave propagates through the whole volume of the Earth. This poses a computational challenge, as no numerical solver can be simulated in infinite domains. Therefore, when numerically simulating wave propagation, a decision has to be made on where to artificially truncate the domain Ω , simultaneously introducing boundary conditions.

While choosing Neumann boundary conditions significantly simplifies computations, it also introduces artificial reflection, as waves hitting the boundary bounce back with the same speed and wavelength and a change of direction. This reflection does not exist in the unbounded real life medium and can interfere with the simulated data, rendering the solver inaccurate despite it being mathematically correct.

Initially, one might consider two solutions to this problem. As the wave propagates, its amplitude attenuates with distance due to geometric spreading. Thus, making the domain Ω sufficiently large makes the waves arrive with less energy to the boundary and reflect in a less intense way. By making Ω big enough, we can

also prevent the wave of hitting the boundary entirely within the simulation time window. However, this approach increases computational cost by requiring many additional elements, and is not very effective when the time interval is large.

Alternatively, another way to solve this problem could be to define the boundary conditions in a way that waves do not reflect when hitting Γ . These boundary conditions are called Absorbing Boundary Conditions (ABCs) [14]. The main problem with ABCs is that they only effectively absorb waves in a particular direction (usually normal incidence), which makes them only applicable in 1-D simulations.

In light of this problem, in 1994 Berenger introduced the Perfectly Matched Layer (PML) [15]. Instead of absorbing boundary conditions, Berenger proposed an absorbing boundary layer, an added artificial layer that would attenuate the waves exponentially so when they eventually hit the boundary, the reflection carried very low energy and would not affect the system.

As a wave changing media also causes reflections, the special property of PML is that it is designed so that the interface of the wave, where the medium changes, does not cause a reflection.

The idea behind PML is to change the PDE instead of changing the medium, attenuating the wave while not causing reflection. By assuming the space far from the region of interest to be linear, homogeneous, and time-invariant, the radiating solutions can be expressed as a superposition of plane waves [16]:

$$u(\mathbf{x}, t) = \sum_{\mathbf{k}, \omega} U_{\mathbf{k}, \omega} e^{i(\mathbf{k} \cdot \mathbf{x} - \omega t)}$$

for constant amplitudes $U_{\mathbf{k}, \omega}$, where ω is the angular frequency, $\mathbf{k}$ the wavevector, and $\mathbf{x} = (x, y)$. Focusing on the x direction, for a fixed angular velocity ω and wavevector $\mathbf{k}$, we can express the planewaves as

$$U(y) e^{i(kx - \omega t)}. \quad (4.6)$$

By assuming k to be positive¹, if instead of evaluating (4.6) for $x \in \mathbb{R}$ we take x to be complex $\mathbb{C}$, the oscillating term starts to decay,

$$e^{i(kx - \omega t)} = e^{i(k\operatorname{Re}(x) + ik\operatorname{Im}(x) - \omega t)} = e^{-k\operatorname{Im}(x)} e^{i(k\operatorname{Re}(x) - \omega t)}.$$

This can be done because (4.6) is an analytic function. Because this analytically continued solution still satisfies the exact same differential equation, no physical discontinuity is introduced, and the transition into this absorbing region is perfectly reflectionless. This change is done via a *complex coordinate stretching*, which can be summed up by a transformation on the derivatives,

$$\frac{\partial}{\partial x} \rightarrow \frac{1}{1 + i \frac{\sigma_x(x)}{\omega}} \frac{\partial}{\partial x},$$

¹Check [16] for details.

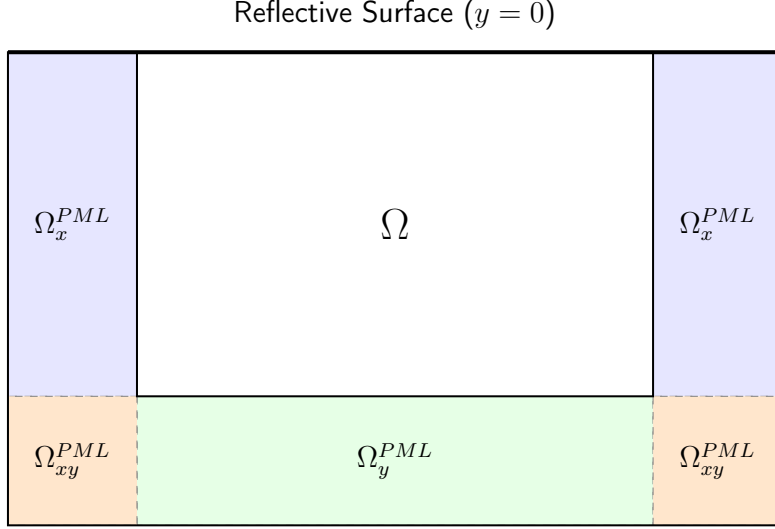


Figure 9: Diagram of a domain Ω extended with PML's depicted by the coloured regions. The reflective surface represents the interface between the sea and atmosphere.

for some specific function σ_x . Applying this transformation inside Ω_x^{PML} introduces the decay for planewaves in the x direction. Similarly, substituting y for x yields the PML in the y direction.

We implement a particular type of PML proposed in [17] called the Convolutional Perfectly Matched Layer (CPML) due to its ease of implementation, better absorption of waves at grazing angles, and overall improved stability [18].

Assuming the PML region is homogeneous, the wave equation transforms into [18]

$$\rho(\mathbf{x}) \frac{\partial^2 u(\mathbf{x}, t)}{\partial t^2} - \nabla \cdot (\mu(\mathbf{x}) \nabla u(\mathbf{x}, t)) = f(\mathbf{x}, t) + f^{PML}(\mathbf{x}, t), \quad (4.7)$$

where

$$f^{PML}(\mathbf{x}, t) = \begin{cases} \mu \left(\frac{\partial \psi_x}{\partial x} + \zeta_x \right) & \text{if } \mathbf{x} \in \Omega_x^{PML} \\ \mu \left(\frac{\partial \psi_y}{\partial y} + \zeta_y \right) & \text{if } \mathbf{x} \in \Omega_y^{PML} \\ \mu \left(\frac{\partial \psi_x}{\partial x} + \frac{\partial \psi_y}{\partial y} + \zeta_x + \zeta_y \right) & \text{if } \mathbf{x} \in \Omega_{xy}^{PML} \\ 0 & \text{if } \mathbf{x} \in \Omega. \end{cases}$$

The corresponding PML regions are those represented in Figure 9.

Discretizing time, ψ_i and ζ_i for $i = x, y$ are defined recursively as

$$\begin{cases} \psi_i(\mathbf{x}, t_n) = a_i(\mathbf{x}) \psi_i(\mathbf{x}, t_{n-1}) + b_i(\mathbf{x}) \frac{\partial u(\mathbf{x}, t_n)}{\partial i} \\ \zeta_i(\mathbf{x}, t_n) = a_i(\mathbf{x}) \zeta_i(\mathbf{x}, t_{n-1}) + b_i(\mathbf{x}) \left(\frac{\partial^2 u(\mathbf{x}, t_n)}{\partial i^2} + \frac{\partial \psi_i(\mathbf{x}, t_n)}{\partial i} \right), \end{cases}$$

where the coefficients $a_i(\mathbf{x}), b_i(\mathbf{x})$ are calculated with

$$a_i(\mathbf{x}) = e^{-(\sigma_i(\mathbf{x}) + \alpha_i)\Delta t}, \quad b_i(\mathbf{x}) = \frac{\sigma_i(\mathbf{x})}{\sigma_i(\mathbf{x}) + \alpha_i} (a_i(\mathbf{x}) - 1).$$

Here, α_i is a constant positive parameter used for complex frequency shifting, and σ_i is the spatial damping profile. Following [19], we compute σ_i as:

$$\sigma_i(\mathbf{x}) = \sigma_i^{\max} \left(\frac{l(\mathbf{x})}{d} \right)^2, \quad \sigma_i^{\max} = \frac{3v_p}{2d} \log \left(\frac{1}{R} \right),$$

where $l(\mathbf{x})$ is the depth within the PML measured from the inner domain boundary and d the thickness of the PML in direction i . R is the reflection coefficient and v_p the wave speed in the PML region.

Because the governing PDE has been modified, the weak formulation must be updated to incorporate the CPML terms,

$$\int_{\Omega} v \rho \ddot{u} \, dA + \int_{\Omega} \mu \nabla v \cdot \nabla u \, dA = \int_{\Omega} v f \, dA + \int_{\Omega} v f^{PML} \, dA.$$

The approximations used to calculate the integral of f^{PML} are the same as described throughout this section. Applying integration by parts and assuming the PML elements to be constant rectangles aligned with the Cartesian axes, for $v = \ell_i(\xi)\ell_j(\eta)$ the contribution of f^{PML} to the weak form approximates to

$$\left\{ \begin{array}{l} \int_{\Omega^e} v \mu (\zeta_x + \zeta_y) \, dA \simeq w_i w_j |\mathcal{J}_{ij}^e| \mu_{ij}^e (\zeta_{x,ij}^e + \zeta_{y,ij}^e) \\ \int_{\Omega^e} \frac{\partial v}{\partial x} \mu \psi_x \, dA \simeq \sum_{l=0}^p w_l w_j |\mathcal{J}_{lj}^e| \mu_{lj}^e \left(\ell'_i(\xi_l) \frac{2}{\Delta x} \right) \psi_{x,lj}^e \\ \int_{\Omega^e} \frac{\partial v}{\partial y} \mu \psi_y \, dA \simeq \sum_{k=0}^p w_i w_k |\mathcal{J}_{ik}^e| \mu_{ik}^e \left(\ell'_j(\eta_k) \frac{2}{\Delta y} \right) \psi_{y,ik}^e \end{array} \right.$$

where Δx (resp. Δy) is the length of Ω^e in the x -direction (resp. y -direction).

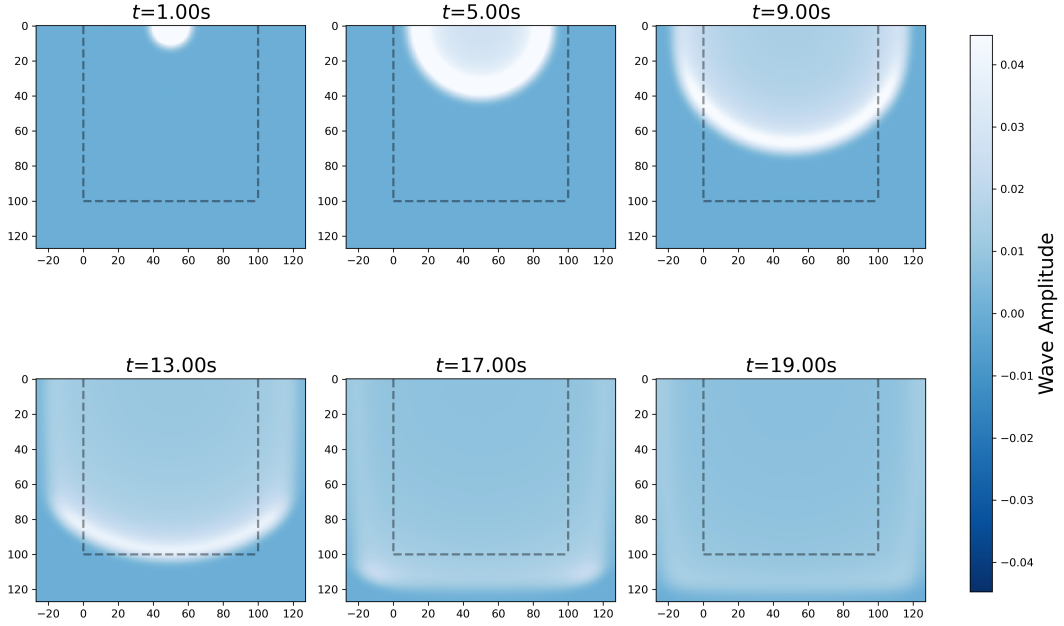


Figure 10: Simulation of a pulse with PML. The interior of the dashed lines represent the domain Ω , and the exterior the PML region.

Typically, the PMLs are extended sideways symmetrically and on the bottom of the computational domain Ω as illustrated in Figure 9. The top of Ω represents the interface between the sea and atmosphere and is left as a reflective boundary. The thickness of the PML (established by the number of elements) determines how strong the absorption is: adding more elements gives more time for the wave to decay at the cost of increased computational overhead.

In Figure 10, we present the results of simulating a pulse using a boundary padded with 8 additional PML layers. We can observe that when the wave enters the PML region—the exterior of the dashed lines—it starts to attenuate and eventually reaches the outer boundary of Ω^{PML} without energy, causing the reflection to not affect the simulation.

Observing the later time steps (e.g., $t = 17.00s$ and $t = 19.00s$), one might notice a slight residual amplitude trailing behind the main wavefront. This is not a numerical artifact or a PML failure, but a characteristic of the wave equation in 2-D. Unlike 1-D or 3-D waves, 2-D waves leave behind a slowly decaying tail [20]. Therefore, the PML successfully absorbs the primary wavefront while correctly

preserving this physically accurate 2-D tail.

4.8 Global Assembly and Implementation Details

The global assembly of the system is the same as explained in subsection 3.6. In fact, as the assembly was done using global indices, as long as they are correctly defined, the method described in subsection 3.6 also works for any n -dimensional version of the problem.

Using the notation in Subsection 4.6, the total nodes in the global grid when `MESH_MODE=0` is

$$T_{\text{nodes}} = (n_x p + 1)(n_y p + 1).$$

For a grid with, for example, $n_x = 10$ and $n_y = 5$ and $p = 5$, the total number of nodes is $T_{\text{nodes}} = 1326$, which would make the stiffness matrix K have $1326 \times 1326 = 1758276$ coefficients. Even though the number of elements is fairly low (50), the stiffness matrix is quite large. Moreover, most of the entries are 0 as the K_{ij} only account for the «interactions» between neighbouring nodes. For simulations with a big number of elements, the matrix exceeds standard memory and cache capacities, rendering the computational process severely inefficient.

To prevent this, the local K^e are not summed to obtain a global K , rather, they are stored in an array of matrices. This way, we are able to store all the non-zero values of K in a much smaller quantity of memory; using this method, instead of storing 1758276 entries, we store $50 \times 36 \times 36 = 64800$ entries, almost 30 times less.

We have this in mind when assembling the global system, as we cannot perform the global multiplication Ku . Instead, we perform local multiplications $K^e u^e$ where u^e corresponds to the nodes in element e in local indices and later assemble the system using global indices.

In the 1 dimensional case, we first computed the u 's for all the generations and then wrote the results in a `.csv`. This makes the u a matrix of size `time_points` $\times$ T_{nodes} . Due to the large quantity of nodes, this cannot be done in 2 dimensions, as doing it this way quickly overflows the memory. Instead, we store 3 generations of u which are needed to compute each time step, and write the results before calculating the next generation of u 's. While frequent disk I/O operations introduce a computational bottleneck, this strategy is strictly necessary to satisfy hardware memory constraints.

5 Validation and Results

In this section we validate the method by running and analysing the implementation of the SEM in different functions and domains. Subsection 5.1 validates and compares the 1-D SEM with a FD solver. In subsection 5.2 we analyse the convergence of the 2-D SEM in two different cases, and finally, we run a simulation in a real-life scenario using the Marmousi model.

For simulations 5.1.1, 5.2.1, and 5.2.2 we use the *Method of Manufactured Solutions*, which consists of taking a function u and plug it in the wave equation to obtain the source f . We then run the solvers for different parameters and compare the numerical results with the analytical solution u . Specifically, we use the Mean Absolute Error (MAE) between the numerical and analytical points in the last instant of the simulation.

The number of time points used was calculated using the Courant-Friedrichs-Lewy (CFL) condition, $C = c \frac{\Delta t}{\Delta x}$ [21], where C was kept constant inside each simulation, Δx is taken as the minimum distance between two nodes, and c is the wave speed, also fix. In the reference domain Λ^2 , the minimum distance between nodes scales as $\mathcal{O}(p^{-2})$ [7], and therefore the closest nodes are at a distance of $\mathcal{O}(d_\Omega p^{-2})$, where d_Ω is the smallest distance between two vertex nodes in Ω .

All numerical simulations and performance benchmarks presented in this section were executed on a machine equipped with an Intel Core i5-10210U CPU @ 1.60 GHz (4 physical cores, 8 threads) and 8 GB of RAM, running a 64-bit Windows operating system. The numerical solver was implemented in C and compiled using GCC with -O3 optimization.²

5.1 Implementation in 1D

5.1.1 Simulation 1

The function used for this simulation was

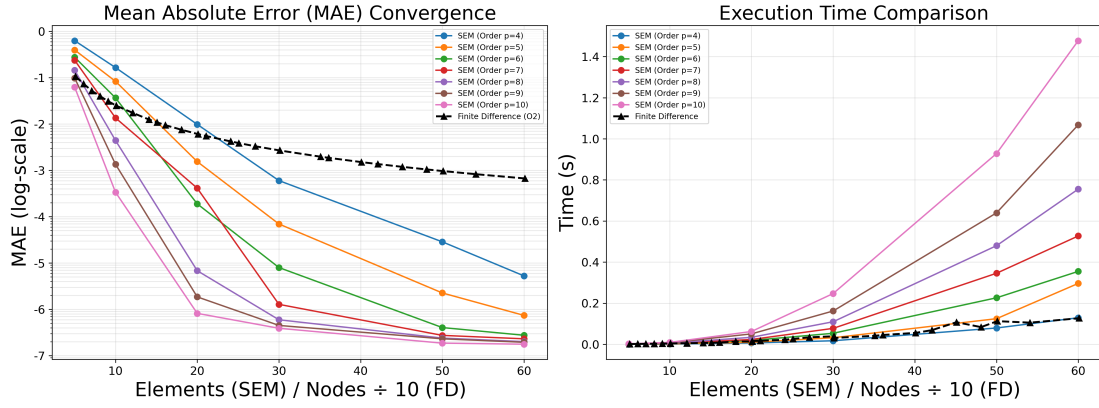
$$u(x, t) = e^{\sin(x)} \cos(t) \sin\left(\frac{\pi x}{L}\right),$$

in the domain $\Omega = [0, L]$, where $L = 50$, and the time interval $I = [0, 3]$. We ran the simulation for $p = \{4, \dots, 10\}$ and $\{5, 10, 20, 30, 50, 60\}$ elements. The number of nodes in each simulation using the FD solver corresponded to the number of nodes in a SEM simulation.

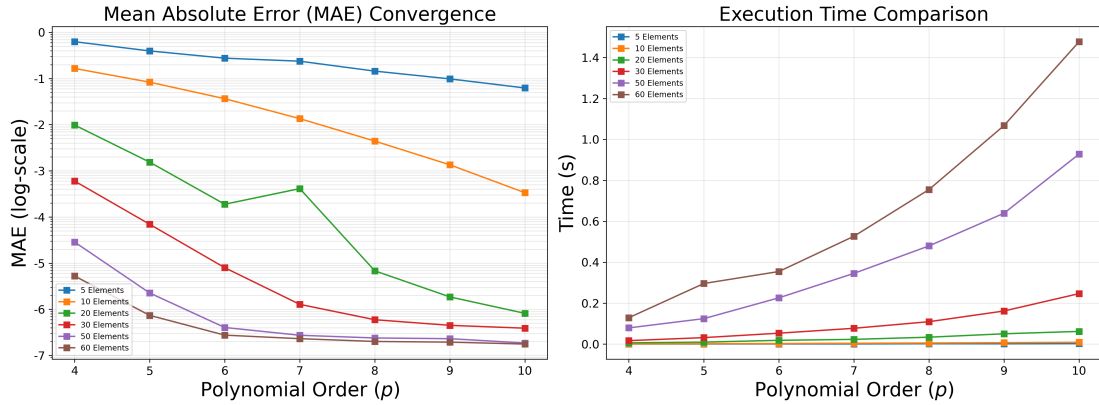
The results, shown in Figure 11, validate the SEM implementation in 1 dimension. The top Figure 11a represents the order fixed and the number of elements increasing (h -refinement) while the bottom Figure 11b represents fixed elements

²Link to the Github implementation.

5.1 Implementation in 1D



(a) h -refinement: Convergence and execution time across increasing element counts.



(b) p -refinement: Convergence and execution time across increasing polynomial orders.

Figure 11: Comparison of numerical accuracy (MAE) and computational cost (execution time) between the 1-D SEM and a standard FD solver. The SEM demonstrates superior exponential convergence, reaching an error floor of $\sim 10^{-6.5}$ significantly faster than the FD scheme. Note that the FD node count is scaled proportionally to match the number of nodes in the SEM solver.

and increasing order (p -refinement). The numerical solution clearly converges to the analytical one both increasing the order, and the number of elements. Comparing it with the FD solver, the MAE is magnitudes of order lower in the SEM solver and decays much faster to the solution, demonstrating spectral/exponential convergence. However, we do see a minimum error of $\sim 10^{-6.5}$, probably due to floating-point error.

The plots on the right of Figure 11 show that the FD scheme remains computationally inexpensive compared to the SEM scheme, although the execution time is fairly low even in the slowest case, peaking at ~ 1.5 seconds.

Therefore, while both methods converge to the solution, the SEM does in a much more efficient way. Comparing solutions, the SEM simulation achieves an error of $\sim 10^{-5.5}$ in 0.125 seconds when $p = 4, n_e = 60$, while in a similar time (0.124 seconds), the FD solution achieves an error of $\sim 10^{-3.4}$. Furthermore, increasing the order to $p = 7$, and $n_e = 30$, the error is of $\sim 10^{-5.9}$ and the execution time is 0.07 seconds. By comparing execution time versus accuracy, we can conclude that the SEM works best when the order is $p = 6, 7$.

5.2 Implementation in 2D

5.2.1 Simulation 1

In Figure 12 we see the results of plotting the function

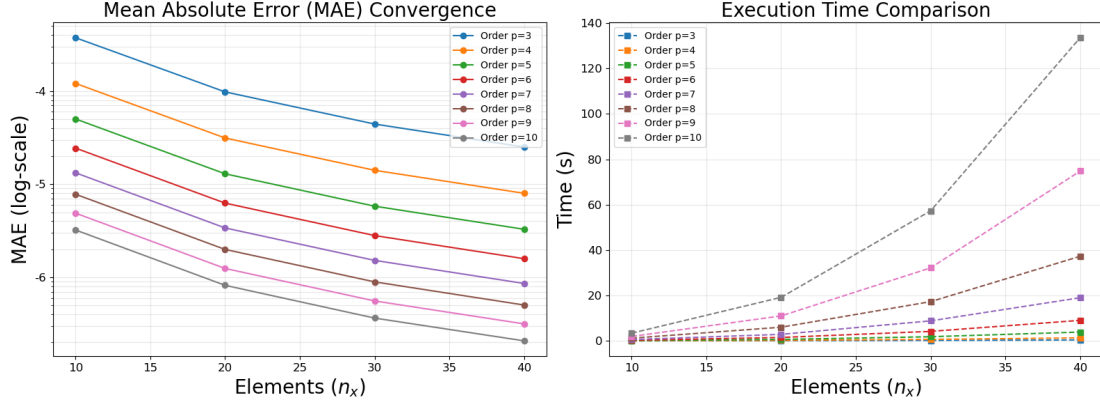
$$u_1(x, y, t) = \sin\left(\frac{\pi}{L_x}x\right) \sin\left(\frac{\pi}{L_y}y\right) \sin(t),$$

in the domain $\Omega = [0, L_x] \times [0, L_y]$ where $L_x = 10$, and $L_y = 6$, and the time interval $I = [0, 10]$, which describes a standing wave fixed on the boundary $u_1|_{\partial\Omega} = 0$. The mesh was initialized with `MESH_MODE=0` so the elements are equidistant in the x direction and in the y direction. The number of elements in the y direction, n_y , is the integer part of $0.6n_x$. We ran the simulation for $p = \{3, \dots, 10\}$ and $n_x = \{10, 20, 30, 40\}$.

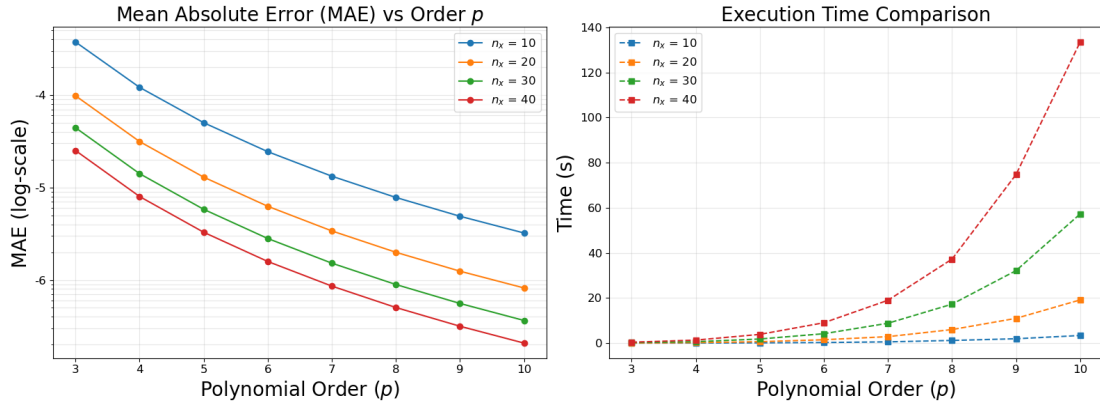
As we can see from the left graphs in Figure 12 the numerical solution properly converges to the analytical one, obtaining a MAE of as low as $\sim 10^{-6.7}$ in the case where $n_x = 40$ and $p = 9$. Moreover, increasing both p and n_x decreases the error.

While the higher order and number of elements provides superior accuracy, the execution time comparison, right graphs of Figure 12, shows that there is a trade-off between accuracy and execution time. Particularly, we see that the time execution for higher orders $p \geq 7$ is magnitudes higher compared to lower orders. For example, we highlight the case where $p = 6, n_x = 30$ and compare it to $p = 10, n_x = 40$. While the error only decreases by approximately one order of magnitude, the execution time increases by $\times 31$.

5.2 Implementation in 2D



(a) h -refinement: Convergence and execution time across elements (n_x).



(b) p -refinement: Convergence and execution time across polynomial orders (p).

Figure 12: Performance and error analysis of the 2-D SEM applied to the standing wave u_1 . The method demonstrates rapid exponential convergence, achieving a Mean Absolute Error (MAE) near $10^{-6.7}$ for the finest grids. However, the execution time tracking (right column) reveals a severe computational bottleneck for higher polynomial orders ($p \geq 7$), driven by the $\mathcal{O}(p^4)$ scaling of the local matrix-vector products evaluated at each time step.

By using profiling tools we can see how the execution time is divided among the steps of the algorithm. In particular, we used the Linux profiling tool `perf` to check where was the main bottleneck of the program. We found that the most expensive part was the computation of the product Ku_{t_n} , repeated every time step. Recall that we defined local K^e 's to save space, and computed $K^e u_{t_n}^e$ for each element. The matrices K^e are of size $(p+1)^2 \times (p+1)^2$, and thus the product is of order $\mathcal{O}(p^4)$. This operation being performed for every element and every time point becomes extremely expensive. While other operations, such as the computation of the coefficients of the K^e , are more expensive, they are performed once and account for a very small chunk of the codes resources. This would become even more noticeable when running the solver in 3 dimensions, as the computation would be triple indexed and would be of order $\mathcal{O}(p^6)$.

In light of this fact, the main objective should be to optimize the computation of Ku . As operations between rows are independent, the loops could be easily parallelized to obtain an almost $\times N$ speed up, where N is the number of cores/processes.

Since the computations also depend on the number of time points in the integration scheme, one could also use higher order methods in the time dimension instead of a basic finite difference scheme, such as Runge-Kutta methods, to decrease the number of time points needed.

5.2.2 Simulation 2

In the next simulation we also take a standing wave as in the previous Example 5.2.1 and modify it. We take u_2 to be

$$u_2(x, y, t) = \sin\left(\frac{100\pi}{L_x}x\right) \sin\left(\frac{100\pi}{L_y}y\right) \sin(t),$$

in the domain $\Omega = [0, L_x] \times [0, L_y]$ where now $L_x = 10000$, $L_y = 6000$. The rest of the simulation parameters are the same as in Example 5.2.1.

We see from Figure 13 that the results are correct, in the sense that the method converges to the analytical solution, but not as accurately as in the previous simulation.

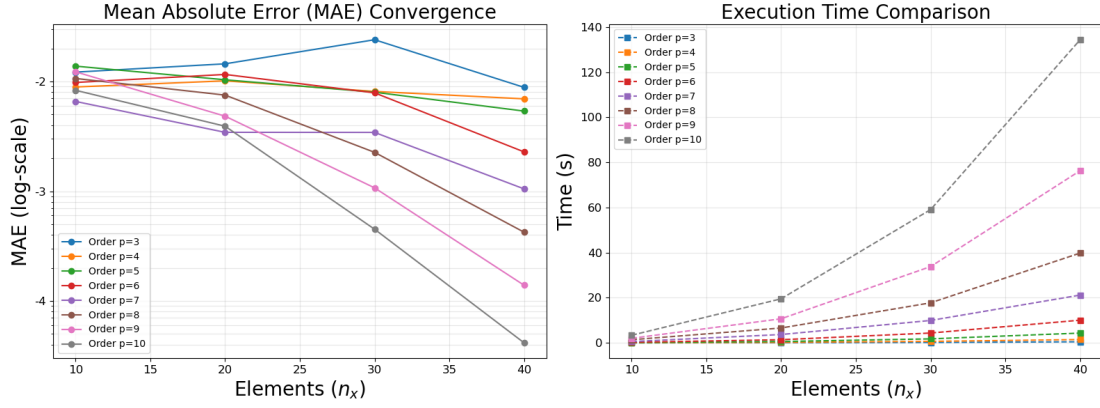
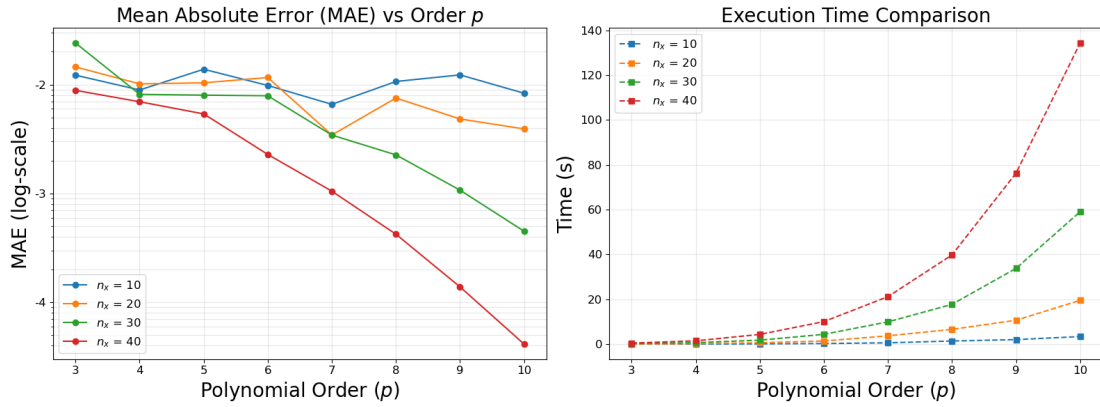

 (a) h -refinement: Convergence and execution time across elements (n_x).

 (b) p -refinement: Convergence and execution time across polynomial orders (p).

Figure 13: Performance and error analysis of the 2-D SEM applied to the high-frequency standing wave u_2 . The top graphs show h -refinement while the bottom graphs show p -refinement. While the method successfully converges, the MAE is noticeably higher than in the low-frequency test case (Figure 12). The results reinforce that for highly oscillatory waves requiring a massive number of nodes, h -refinement achieves similar target accuracies much faster than p -refinement due to the $\mathcal{O}(p^4)$ bottleneck in the local matrix-vector products.

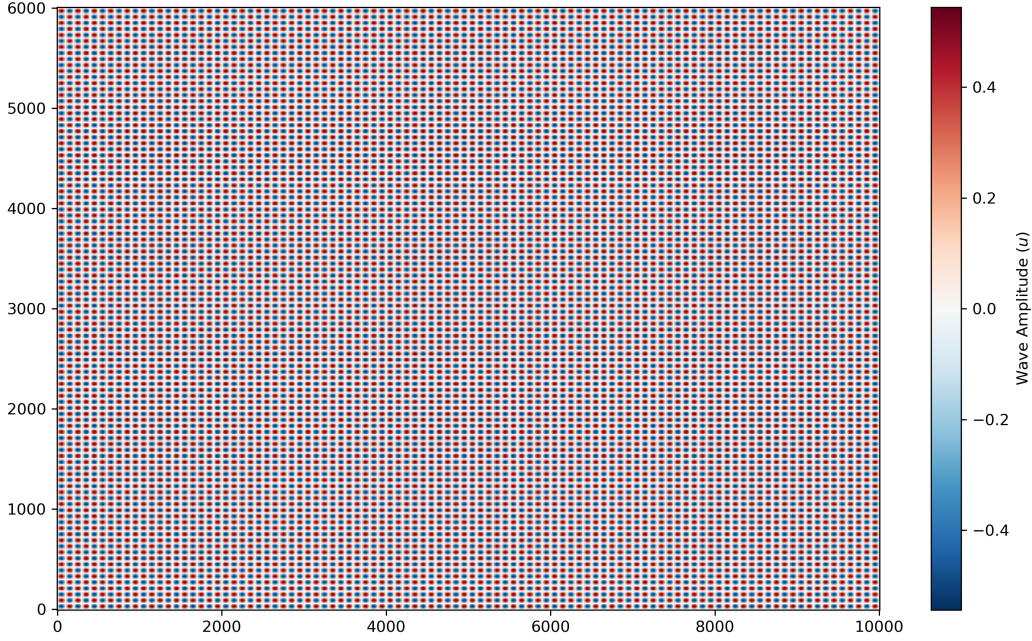


Figure 14: Analytical spatial distribution of the high-frequency standing wave u_2 evaluated at $t = 10$. The scale of the domain ($10,000 \times 6,000$ m) combined with the high frequency of the sinusoidal terms creates rapid spatial oscillations (short wavelengths). If the PPW is insufficient the solver is not able to capture the oscillations, explaining the reduced accuracy in the low node and polynomial order cases.

Figure 14 presents the analytical solution u at $t = 10$ s. The wave exhibits rapid spatial oscillations in both the x and y directions due to the large scaling factors within the sinusoidal terms. Because the spatial discretization parameters were not scaled to match this frequency, the distance between nodes becomes too coarse to capture these oscillations, causing the numerical solution to lack spatial resolution. For context, the analytical wavelength in the x -direction is $\lambda_x = 200$. In the lowest-resolution scenario ($p = 4$, $n_x = 10$), the maximum distance between nodes is approximately 320. At this resolution, multiple waves fit between adjacent nodes, making it physically impossible to capture the oscillations. In the highest-resolution configuration ($p = 10$, $n_x = 40$), the minimum nodal distance is approximately 35. In wave simulations, *Points-Per-Wavelength* (PPW) measures how many nodes fit inside a distance of a wavelength, and it is a crucial measure that determines if a simulation is able to capture the physics of the wave. If the PPW is insufficient, dispersion errors appear and can render the simulation useless. In these cases, a finer mesh is needed to increase the PPW.

As previously established regarding the $\mathcal{O}(p^4)$ computational bottleneck, h -

refinement is substantially more efficient than p -refinement for large-scale, high-frequency grids. We ran the simulation for $p = 6$ and $n_x = 120$ and ran it against the best performing parameters ($p = 10$, $n_x = 40$) and found a similar accuracy (MAE of 3.55×10^{-5} as opposed to 4.14×10^{-5}) while the runtime was significantly higher in the high order case (134 seconds) against the high element count case (98 seconds).

5.2.3 Simulation 3: The Marmousi Model

The Marmousi model [22, 23] is a 2-D grid example developed by the Institut Français du Pétrole (IFP) used as a benchmark to evaluate solvers. It was originally constructed by the IFP to challenge and test seismic wave inversion and imaging algorithms on complex conditions. Based on a section in the Cuanza Basin, it is a 9575×4025 meter grid with 25m separation between nodes.

The complexity of the Marmousi model comes from the many different layers, which cause wave reflections, steep dips and strong velocity changes. The data was obtained having in mind that conventional inversion frameworks were not good enough to capture the subtleties in the model. Due to this, it has become the reference model to test FWI solvers [24, 25].

To use this model, our solver reads a `.xyz` file that contains the velocity at each node. Recall that the velocity of the wave is related to the density and bulk modulus using the following formula,

$$c(x, y) = \sqrt{\frac{K(x, y)}{\rho(x, y)}}.$$

Therefore, by setting the density to be 1 everywhere, we can obtain the spatial stiffness with the formula $K(x, y) = c(x, y)^2$.

However, the grid only contains the velocity in the nodes, which are spaced 25m apart. GLL points are not evenly spaced, so in order to solve the wave equation in the Marmousi model, we need to infer the velocity in any (x, y) coordinate of the grid. To do this, we use a bilinear interpolation scheme, explained in further detail in A.2, which uses the known values of the velocity in the nodes to approximate the velocity anywhere in the grid.

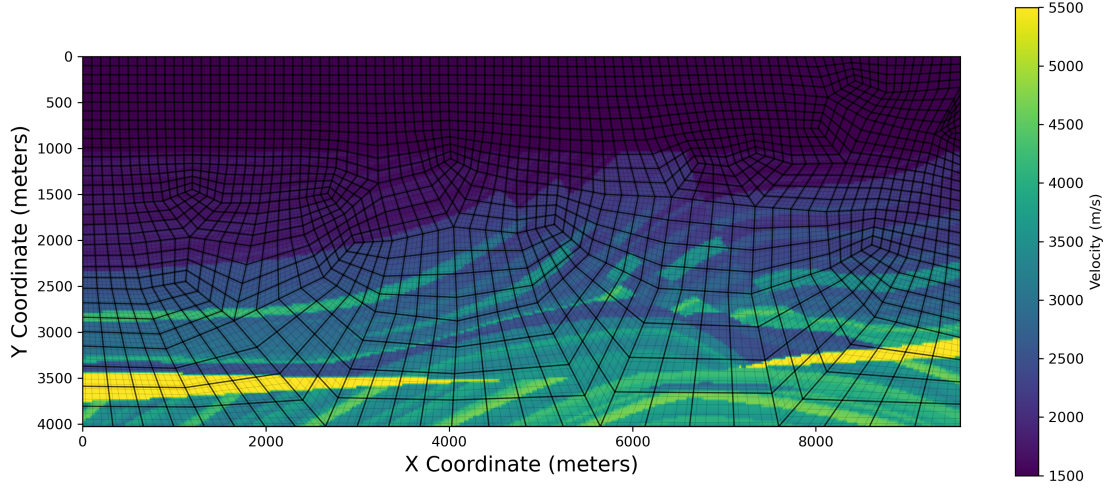


Figure 15: Overlay of the spatially graded computational mesh onto the Marmousi velocity model. To optimize computational resources while strictly preventing numerical dispersion, the physical element sizes are adapted to the local wave speed. Smaller elements are concentrated in the shallow, low-velocity regions (~ 1500 m/s) where wavelengths are shortest, whereas larger elements are utilized in the deeper, high-velocity formations (~ 5500 m/s) where the spatial resolution requirements are less strict.

To create the mesh we use `MESH_MODE=1`, as SALOME has the option to control the size of the elements in a scalar field. We are interested in having smaller elements where the wavelength λ is small and finer measurements are required.

A wave propagating through different materials does not change its frequency f [26], and using the relation $c = f\lambda$, where λ is the wavelength, we deduce that c and λ are proportional. Therefore, we need the mesh to be finer on the regions where the velocity is lower to satisfy the spatial sampling requirement and prevent numerical dispersion (ensuring an adequate PPW). For marine scenarios or benchmarks like the Marmousi model, this leads to a vertically graded mesh: highly dense in the shallow water and sediment area where wavelengths are shortest, and wider at greater depths where higher velocities stretch the wavefronts, as illustrated in Figure 15.

While this synthetic study employs an ideal, non-attenuative forward solver where frequency is preserved, this multi-scale mesh design also aligns with real-world seismic acquisition. In field data, the Earth acts as a low-pass filter that absorbs high frequencies with depth. Furthermore, this variable resolution perfectly matches the ray-path coverage of marine acquisition geometries: the high density and diverse angular coverage of seismic rays in the shallow subsurface justify a highly refined inversion grid, whereas the poorer illumination and wider

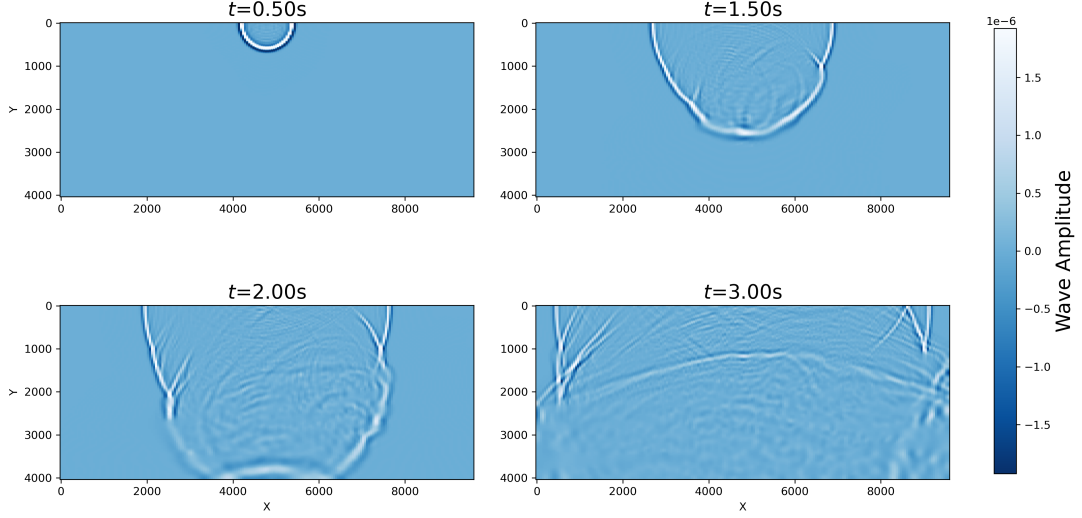


Figure 16: Time evolution of a Ricker wavelet propagating through the highly heterogeneous Marmousi model. The successive snapshots ($t = 0.5, 1.5, 2.0$, and 3.0 seconds) demonstrate the complex scattering, refractions, and reflections caused by the varying geological layers. As the wave penetrates deeper, the wavefront distorts significantly, highlighting the solver’s capability to accurately capture the subtleties of the model.

ray paths at depth naturally limit the resolvable structural details. Therefore, although attenuation and ray decay are omitted in these simulations, coarsening the mesh at depth keeps the model computationally efficient and follows standard geophysical practices in propagation and inversion.

In FWI schemes, it is important to have an accurate source wavelet. Several types are typically used [27], among them the *Ricker wavelet/pulse*. The Ricker wavelet, named after Norman Ricker (1896-1980), is the negative second derivative of a Gaussian function,

$$w(\tau) = (1 - 2\pi^2 f_p^2 \tau^2) e^{-\pi^2 f_p^2 \tau^2},$$

where f_p is the peak frequency of the wavelet.

For a realistic study case, we simulate a Ricker pulse in the Marmousi model. We do this by making the source term f represent a Ricker pulse centred at the surface,

$$f(x, y, t) = (1 - 2\pi^2 f_p^2 t^2) e^{-\pi^2 f_p^2 t^2} e^{-\frac{d^2}{a}},$$

where d is the distance between a point and the source of the pulse $(\frac{9575}{2}, 0)$, and we set $a = 100$. The second exponential is an attenuation term that depends on the distance so the pulse is not concentrated on a single point.

We ran the simulation for $t = 5$ seconds and plotted different time frames in Figure 16. We used the mesh in Figure 15, which contains 2024 elements of order $p = 5$, generating 51061 nodes and set 10000 time steps, keeping the simulation stable.

In the simulation frames we see the wave passing through regions with different properties, causing reflections and non-uniform propagation of the wavefront.

6 Conclusions and Further Work

In this thesis, we successfully developed and validated a 1-D and 2-D Spectral Element Method (SEM) forward solver for acoustic wave propagation, specifically designed for Full Waveform Inversion (FWI) frameworks. The validation results confirm the theoretical advantages of this approach: in the 1-D case, the SEM solver outperformed the standard FDM solver, requiring a significantly lower Points-Per-Wavelength (PPW) count while achieving superior accuracy and computational efficiency. In the 2-D case, the solver accurately converged to the exact analytical solutions, showcasing the flexibility of the SEM implementation through both h -refinement (increasing elements) and p -refinement (increasing polynomial order).

While the SEM solver successfully achieved spectral accuracy, increasing the polynomial order p exposed a significant execution time bottleneck caused by the $\mathcal{O}(p^4)$ scaling of the local matrix-vector products. To mitigate this and maximize efficiency, we integrated a non-uniform mesh generation framework using the SALOME platform. This allowed us to dynamically adapt the element size to the local physical properties of the domain, using larger elements where low PPW resolution was sufficient and thus reducing the global computational load. The solver was successfully validated on the Marmousi model, a highly detailed and heterogeneous benchmark model for which this spatially graded mesh proved ideal.

To accurately approximate unbounded geological media, we implemented Convolutional Perfectly Matched Layers (CPML). This boundary condition successfully absorbed outgoing waves with high efficiency, effectively simulating infinite domains within a finite grid.

Future work should primarily focus on computational optimization. As identified in Section 5, the local matrix-vector product $K^{ee}u^e$ constitutes the main bottleneck. Because these elemental products are independent, they are highly parallelizable using OpenMP, MPI, or GPU acceleration (CUDA). As these spatial products are evaluated at every time step, upgrading the temporal discretization to higher-order integration schemes, such as 4th-order Runge-Kutta or Local Time-Stepping (LTS), would allow for larger Δt increments and dramatically reduce simulation runtimes.

The physical accuracy and applicability of our model are currently constrained by the 2-D acoustic approximations detailed in the introduction. To fully bridge the gap between synthetic simulations and real-world seismic data, it is essential to update the computational framework to a fully 3-D solver. Furthermore, future versions of the solver should re-introduce the shear modulus $\mu \neq 0$ to model S-waves within the full elastic wave equation. Additionally, incorporating viscoelasticity would allow the solver to model intrinsic energy dissipation (attenuation),

accurately simulating how real Earth materials absorb high-frequency wave energy over depth.

Ultimately, the forward solver developed in this thesis represents the computational engine of the Full Waveform Inversion process. To close the FWI loop, future work must focus on implementing the adjoint wave equation. By back-propagating the data residuals from the receivers and convolving them with the forward wavefield, the gradient of the misfit function can be computed via the Adjoint State Method. The computation of the gradient together with a gradient-based scheme (such as NLCG) would complete the FWI framework, allowing the model to iteratively update its parameters to achieve a high resolution seismic image.

The need for an efficient solver also relates to the trade-off between the iterative process of updating the parameters and the initial approximation to the solution. The number of iterations needed to obtain an accurate seismic image depends heavily on the initial approximation, as a good initial guess allows the parameters to converge faster to the true value. However, obtaining this initial guess can be challenging, and while it may take the FWI framework fewer iterations to converge, the high cost of the initial guess can overwhelm the overall computational cost. Therefore, the efficiency of the solver and the FWI framework play an important role in what the accuracy of the initial guess should be for the model parameters.

References

- [1] Tarantola A. Inversion of seismic reflection data in the acoustic approximation. *Geophysics*. 1984;49(8):1259-66. Available from: <https://doi.org/10.1190/1.1441754>.
- [2] Virieux J, Operto S. An overview of full-waveform inversion in exploration geophysics. *Geophysics*. 2009;74(6):WCC1-WCC26. Available from: <https://doi.org/10.1190/1.3238367>.
- [3] Carcione JM. *Wave Fields in Real Media: Wave Propagation in Anisotropic, Anelastic, Porous and Electromagnetic Media*. 4th ed. Elsevier Science; 2022.
- [4] Matthews PC. *Vector Calculus*. Springer Undergraduate Mathematics Series. London: Springer London; 1998. Available from: <https://ebookcentral.proquest.com/lib/uab/detail.action?docID=3073487>.
- [5] Epperson JF. On the Runge Example. *The American mathematical monthly*. 1987 Apr;94(4):329.
- [6] Larson MG, Bengzon F. *The Finite Element Method: Theory, Implementation, and Applications*. vol. 10. 2013th ed. Bengzon F, editor. Heidelberg: Springer Nature; 2013.
- [7] Shen J, Tang T, Wang LL. *Spectral methods: algorithms, analysis and applications*. 1st ed. New York: Springer; 2011.
- [8] Stein EM, Shakarchi R. *Fourier analysis: an introduction*. vol. 1. 1st ed. Princeton, N.J: Princeton University Press; 2003.
- [9] Hafeez MB, Krawczuk M. A Review: Applications of the Spectral Finite Element Method. *Archives of Computational Methods in Engineering*. 2023;30(6):3453-65. Available from: <https://doi.org/10.1007/s11831-023-09911-2>.
- [10] Schuberth B. *The Spectral Element Method for Seismic Wave Propagation* [Master's thesis]. Ludwig-Maximilians-Universität München; 2003.
- [11] Komatitsch D, Tromp J. Introduction to the spectral element method for three-dimensional seismic wave propagation. *Geophysical Journal International*. 1999;139(3):806-22. Available from: <https://onlinelibrary.wiley.com/doi/abs/10.1046/j.1365-246x.1999.00967.x>.

REFERENCES

- [12] Pasquetti R, Rapetti F. Spectral element methods on triangles and quadrilaterals: comparisons and applications. *Journal of computational physics*. 2004;198(1):349–362.
- [13] SALOME Project. SALOME: The Open Source Integration Platform for Numerical Simulation; 2025. Version 9.15.0. Available from: <https://www.salome-platform.org/>.
- [14] Taflov A, Hagness SC. Computational electrodynamics: the finite-difference time-domain method. 3rd ed. Boston;; Artech House; 2005.
- [15] Berenger JP. A perfectly matched layer for the absorption of electromagnetic waves. *Journal of Computational Physics*. 1994;114(2):185-200. Available from: <https://www.sciencedirect.com/science/article/pii/S0021999184711594>.
- [16] Johnson SG. Notes on Perfectly Matched Layers (PMLs); 2021. Available from: <https://arxiv.org/abs/2108.05348>.
- [17] Roden JA, Gedney SD. Convolution PML (CPML): An efficient FDTD implementation of the CFS-PML for arbitrary media. *Microwave and Optical Technology Letters*. 2000;27(5):334-9. Available from: <https://onlinelibrary.wiley.com/doi/abs/10.1002/1098-2760%2820001205%2927%3A5%3C334%3A%3AAID-MOP14%3E3.0.CO%3B2-A>.
- [18] Convolutional Perfectly Matched Layer For Isotropic And Anisotropic Acoustic Wave Equations. vol. 2010 SEG Annual Meeting of SEG International Exposition and Annual Meeting; 2010.
- [19] Drossaert FH, Giannopoulos A. A nonsplit complex frequency-shifted PML based on recursive integration for FDTD modeling of elastic waves [Article]. *Geophysics*. 2007;72(2):T9–T17. Cited by: 166. Available from: <https://www.scopus.com/inward/record.uri?eid=2-s2.0-33947156416&doi=10.1190%2f1.2424888&partnerID=40&md5=0a0defb41adb76954ee5f8c202f935c8>.
- [20] Courant R, Hilbert D. *Methods of Mathematical Physics, Volume II: Partial Differential Equations*. Wiley-Interscience; 1962.
- [21] Courant R, Friedrichs KO, Lewy H. Über die partiellen Differenzengleichungen der mathematischen Physik. *Mathematische Annalen*. 1928;100(1):32-74. Available from: <https://doi.org/10.1007/BF01448839>.

REFERENCES

- [22] Brougois A, Bourget M, Lailly P, Poulet M, Ricarte P, Versteeg R. Marmousi, model and data; 1990. .
- [23] Martin GS, Wiley R, Marfurt KJ. Marmousi2: A renewed elastic synthetic model for imaging and inversion. *The Leading Edge*. 2006;25(2):156-66. Available from: <https://doi.org/10.1190/1.2172306>.
- [24] Choi Y, Min DJ, Shin C. Two-dimensional waveform inversion of multi-component data in acoustic-elastic coupled media. *Geophysical Prospecting*. 2008;56(6):863-81. Available from: <https://doi.org/10.1111/j.1365-2478.2008.00735.x>.
- [25] Jiménez Tejero CE, Dagnino D, Sallarès V, Ranero CR. Comparative study of objective functions to overcome noise and bandwidth limitations in full waveform inversion. *Geophysical Journal International*. 2015;203(1):632-45. Available from: <https://doi.org/10.1093/gji/ggv288>.
- [26] Bedford A, Drumheller DS. *Introduction to Elastic Wave Propagation*. 2nd ed. Cham: Springer Nature Switzerland; 2023.
- [27] Hubral P, Tygel M. Analysis of the Rayleigh pulse. *Geophysics*. 1989 05;54(5):654-8. Available from: <https://doi.org/10.1190/1.1442692>.

Appendices

A Algorithms

A.1 Pseudocode

To better understand the algorithms used in meshing the domain Ω , we write the pseudocode of both initializations. Algorithm 1 corresponds to `MESH_MODE=0`, which takes as input a rectangular domain Ω and divides it into rectangles of constant size. Algorithm 2 corresponds to `MESH_MODE=1`, where an initial grid of quadrilateral elements is meshed by the SALOME software and converted to a SEM mesh by adding the GLL nodes.

Algorithm 1 Structured Rectangular Spectral Grid Initialization

```
1: procedure INITMANUALGRID( $n_x, n_y, p$ )
2:    $N_{rows} \leftarrow n_y p + 1, N_{cols} \leftarrow n_x p + 1$ 
3:   Initialize  $Connectivity[p^2][N_{elements}]$ ,  $LookupTable[N_{rows}][N_{cols}]$  and
      $GlobalNodes$  to empty
4:   Compute GLL points  $z_{0...p-1}$ 
5:   for each Element  $e$  do
6:     Compute element's grid offsets ( $Floor, Door$ )
7:     for  $i \leftarrow 0$  to  $p$  do
8:       for  $j \leftarrow 0$  to  $p$  do
9:          $(\xi, \nu) \leftarrow (z_i, z_j)$ 
10:         $(Idx_{row}, Idx_{col}) \leftarrow (j + Floor, i + Door)$ 
11:        if  $LookupTable[Idx_{row}][Idx_{col}]$  is empty then
12:           $ID_{global} \leftarrow$  next available index
13:           $LookupTable[Idx_{row}][Idx_{col}] \leftarrow ID_{global}$ 
14:           $(x, y) \leftarrow \mathcal{F}_e(\xi, \nu)$ 
15:           $GlobalNodes[ID_{global}] \leftarrow (x, y)$ 
16:        else
17:           $ID_{global} \leftarrow LookupTable[Idx_{row}][Idx_{col}]$ 
18:        end if
19:         $Connectivity[i \cdot p + j][e] \leftarrow ID_{global}$ 
20:      end for
21:    end for
22:  end for
23: end procedure
```

Algorithm 2 High-Order Mesh Generation via Isoparametric Mapping

```

1: procedure MESHCONVERTER(BaseMesh, Order  $p$ )
2:   Load Vertices and Elements from BaseMesh
3:   Compute GLL points  $z_{0\dots p-1}$ 
4:   Initialize Connectivity $[p^2][N_{elements}]$  and GlobalNodes array
5:   for each Element  $e$  do
6:     Retrieve corner vertices  $(v_1, v_2, v_3, v_4)$ 
7:     for  $i \leftarrow 0$  to  $p$  do
8:       for  $j \leftarrow 0$  to  $p$  do
9:          $(\xi, \nu) \leftarrow (z_i, z_j)$  ▷ Reference coordinates
10:         $(x, y) \leftarrow \mathcal{F}_e(\xi, \nu)$  ▷ Map to physical space
11:        if node  $(i, j)$  is on an Element Edge then
12:          if  $(x, y)$  already exists in GlobalNodes then
13:             $ID_{global} \leftarrow$  existing index
14:          else
15:             $ID_{global} \leftarrow$  next available index
16:             $GlobalNodes[ID_{global}] \leftarrow (x, y)$ 
17:            Check and mark if  $ID_{global}$  is on Domain Boundary
18:          end if
19:          else ▷ Internal node
20:             $ID_{global} \leftarrow$  next available index
21:             $GlobalNodes[ID_{global}] \leftarrow (x, y)$ 
22:          end if
23:           $Connectivity[i \cdot p + j][e] \leftarrow ID_{global}$ 
24:        end for
25:      end for
26:    end for
27: end procedure

```

A.2 Bilinear Interpolation

Bilinear interpolation is an algorithm to interpolate the value of a function that is linear on the x and y directions inside a quadrilateral. It is also used to approximate the value of a function f . If the function is continuous, the approximation converges to the real value when making the area of the quadrilateral tend to 0.

To compute the value of f at a given point $P = (x, y)$, we take the 4 nearest nodes Q_{ij} , $i, j = 1, 2$ that define a quadrilateral. Next, we interpolate linearly in the x direction in the upper and lower side of the quadrilateral and use these values to interpolate vertically. The final formula is a degree 2 polynomial in x

and y . In the unit square this polynomial is the following:

$$f(x, y) = f(0, 0)(1 - x)(1 - y) + f(0, 1)(1 - x)y + f(1, 0)x(1 - y) + f(1, 1)xy.$$

In the Marmousi model Example 5.2.3, to be able to apply this formula, we map each of the squares defined by 4 nodes to the unit square.

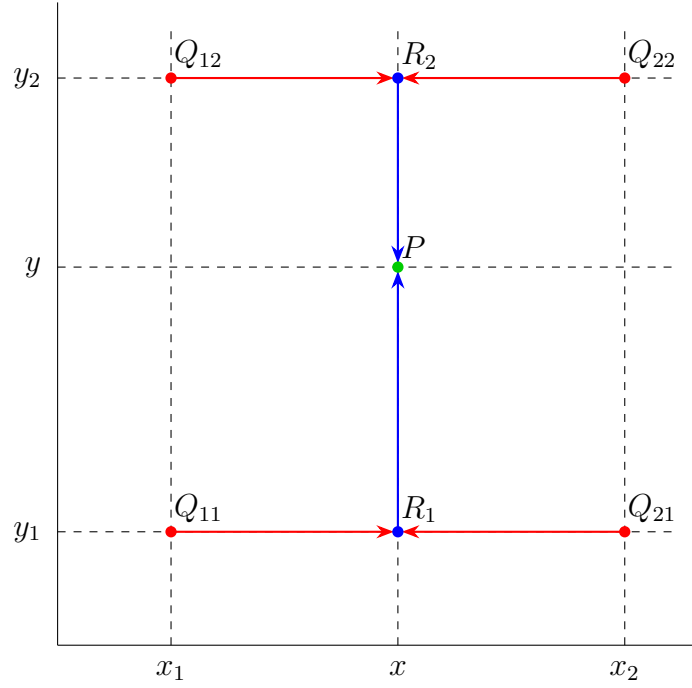


Figure 17: Diagram of a bilinear interpolation scheme.